instituto
superior de
engenharia
de lisboa

Department of Computer Science and Engineering

Remote Lab

50565: Ângelo Filipe Maia Azevedo (a50565@alunos.isel.pt)
50539: António Miguel Alves (a50539@alunos.isel.pt)

Report for Project and Seminar Class
of Computer Science and Computer Engineering BSc

Advisor: Prof. Pedro Miguens Matutino

July 2025

Lisbon School of Engineering

Remote Lab

50565 Ângelo Filipe Maia Azevedo

50539 António Miguel Alves

Advisor: Prof. Pedro Miguens Matutino

Report for Project and Seminar Class of Computer Science and Computer Engineering BSc

July 2025

Abstract

The design, development, implementation, and validation of digital systems require, in addition to simulators, the use of hardware for verification of their implementation in real devices. However, access to these real devices is sometimes restricted, not being available 24h/7. In the current teaching paradigm where face-to-face time tends to reduce remote and autonomous work is increased, it is necessary to create alternatives to the usual model.

The Remote Lab project aims to provide an online laboratories platform, with access to remote hardware. This remote workbench consists of a web application, accessed through a website and provides a dashboard where users can join a laboratory. This is where users can control the remote hardware through a remote terminal. A hierarchy system is implemented to provide different roles, each with their own permissions relative to how users can browse the information provided by the web application.

This project implements the infrastructure to support the configuration, manipulation and visualization of remote hardware. Based on an architecture with back-end (database and Web API) and front-end (Web App, with a dashboard).

Resumo

A conceção, desenvolvimento, implementação, e por fim a validação de sistemas digitais requerem para além dos simuladores, a utilização de hardware para uma verificação da sua concretização em dispositivos reais. No entanto, o acesso a esses dispositivos reais é por vezes restrito, não estando acessíveis 24h/7. No atual paradigma de ensino em que o tempo de contacto visual tende a reduzir, aumentando-se o trabalho remoto e autónomo, é necessário criar alternativas ao modelo habitual.

O projeto Remote Lab tem como objetivo fornecer laboratórios online com acesso a hardware remoto. Este meio remoto consiste numa aplicação web acedida através de um website, que visa fornecer um painel de controlo onde os utilizadores podem aderir a um laboratório. Os utilizadores podem controlar o hardware remoto através de um terminal. Um sistema hierárquico foi implementado para fornecer diferentes funções, cada uma com as suas próprias permissões relativamente à forma como os utilizadores podem navegar pela informação fornecida pela aplicação web.

Este projeto implementa a infraestrutura de suporte à configuração, manipulação e visualização de hardware remoto. Baseado numa arquitetura com back-end (base de dados e Web API) e front-end (Web App, com um dashboard).

Contents

List of Figures **xi**

List of Listings **xiii**

Acronyms **1**

1 Introduction **3**

1.1 Objectives . . . 3

1.2 State of the Art . . . 4

1.3 Document Structure . . . 4

2 Proposed Architecture **7**

2.1 System Users . . . 8

2.2 System Functionalities . . . 8

2.3 System Components . . . 9

3 Database **11**

3.0.1 Core Entities . . . 11

3.0.2 Summary . . . 13

4 Web API **17**

4.1 API Architecture . . . 18

4.2 Roles . . . 22

4.3 Laboratory Sessions . . . 22

4.3.1 Waiting Queue . . . 23

4.3.2 Session . . . 26

4.4 Summary . . . 26

5 Web Application **29**

5.1 Overview . . . 29

5.2 Architecture and Logic Separation . . . 30

5.3 Main Features . . . 30

5.4 Authentication with NextAuth . . . 33
5.5 User Interface . . . 33
5.6 Summary . . . 41

6 Project Organization and Deployment 43
6.1 Project Structure . . . 43
6.1.1 Hardware Integration and Simulation . . . 44
6.2 Deployment . . . 45
6.2.1 Containerization and Orchestration . . . 45
6.2.2 Environment Configuration and Secrets . . . 45
6.2.3 Automation with start.sh . . . 46
6.2.4 Cloudflare Tunneling . . . 46
6.2.5 Nginx as Reverse Proxy . . . 46
6.2.6 Deployment Steps . . . 47
6.3 Build and Continuous Integration and Continuous Deployment (CI/CD) . . . 47
6.4 Summary . . . 47

7 Experimental Results 49

8 Conclusions 51
8.1 Conclusions . . . 51
8.2 Future Work . . . 51

References 56

A Database Documentation 57
A.1 Introduction . . . 57
A.2 Database Overview . . . 57
A.3 Entity-Relationship Model . . . 58
A.4 Entities and Attributes . . . 58
A.4.1 User . . . 58
A.4.2 Token . . . 59
A.4.3 Group . . . 59
A.4.4 Laboratory . . . 59
A.4.5 Hardware . . . 60
A.5 Associations . . . 60

B Domain Configurations 63

List of Figures

2.1 High-level Architecture 7
2.2 Detailed System Architecture 9

3.1 User Entity 11
3.2 Token Entity 12
3.3 Laboratory Entity 12
3.4 Hardware Entity 13
3.5 Group Entity 13
3.6 Entity-Relationship Model (ER Model) 15

4.1 API Architecture Overview 18
4.2 API Detailed Architecture 19
4.3 Comparison between filters and interceptors. Source: Baeldung [1]. 20
4.4 Server-Sent Events Example 23
4.5 Laboratory Waiting Queue Scheme 24
4.6 Wait-Path Scheme 25

5.1 Authentication flow: (a) login screen; (b) Microsoft OAuth login page shown after clicking the login button. 34
5.2 Role selection dropdown 34
5.3 Professor dashboard 35
5.4 Lab creation modal 35
5.5 Interactive modal with validation 36
5.6 Full-screen lab creation 36
5.7 Success message after creation 36
5.8 Lab editing interface 37
5.9 Lab group management 37
5.10 Group creation form 37
5.11 Group created confirmation 38
5.12 Group user management 38
5.13 Lab hardware management 39

5.14 Hardware added to lab 39
5.15 Hardware editing interface 39
5.16 Lab session start 40
5.17 Waiting queue 40
5.18 Account dropdown menu for quick access to profile, settings, and logout. 40
5.19 Detailed account information. 41
5.20 Role change dropdown (admin) 41

7.1 Unit Tests Results 49

A.1 ER Model 58

Listings

4.1 Group Entry System Configuration Example 21
B.1 Domain Configurations 63

Acronyms

API Application Programming Interface

BSc Bachelor of Science

CI/CD Continuous Integration and Continuous Deployment

CRUD Create; Read; Update; Delete

DevOps Development and Operations

ER Model Entity-Relationship Model

FPGA Field-Programmable Gate Array

HTTP HyperText Transfer Protocol

IP Internet Protocol

ISA iLab Shared Architecture

JDBC Java Database Connectivity

JDBI Java Database Interface

JSON JavaScript Object Notation

MAC Media Access Control

MIT Massachusetts Institute of Technology

OAuth Open Authorization

OOP Object-Oriented Programming

RBAC Role-Based Access Control

REST Representational State Transfer

URI Uniform Resource Identifier

Chapter 1

Introduction

In recent years, the need for remote access to laboratory resources has grown significantly, driven by the expansion of online education, increased research collaboration, and the growing complexity of experimental setups. Traditional laboratories often require physical presence, which can limit accessibility and flexibility for students, researchers, and professionals. This limitation has become particularly evident in situations where geographical constraints, time restrictions, or extraordinary circumstances, such as global pandemics, prevent direct access to laboratory facilities.

The project described in this report addresses these challenges by developing a comprehensive solution for remote laboratory access that maintains the quality and integrity of hands-on experimentation while providing the flexibility of remote operation.

1.1 Objectives

Considering these emerging needs, a platform was proposed and implemented to enable secure, efficient, and user-friendly remote access to laboratory equipment and resources. The design of the platform was divided into two distinct phases. In the first phase, a database, an Application Programming Interface (API), and a web application were designed and implemented. This phase also encompassed the deployment architecture of the platform, including containerization, orchestration, and other essential configurations. In the second phase, the communication protocols between the platform and laboratory hardware were designed and implemented.

To design and implement a scalable platform for remote laboratory access, the following main objectives were established:

- Web API to ensure comprehensive user, laboratory, and hardware management;
- Web application that provides an intuitive and user-friendly interface;
- Secure authentication and authorization mechanisms;
- Hierarchical system of roles;

- Robust data persistence through a well-designed database system;
- Remote manipulation capabilities via terminal access to laboratory devices.

Additionally, several optional objectives were identified to enhance the system's functionality:

- Laboratory scheduling and reservation system;
- Real-time visual monitoring of laboratory hardware.

1.2 State of the Art

Numerous initiatives have emerged to provide remote access to laboratory resources, particularly in higher education and research contexts. Pioneering projects such as MIT's iLab Shared Architecture (ISA) [2] and LabShare [3] have demonstrated both the feasibility and substantial benefits of remote laboratories, enabling students and researchers to conduct experiments from anywhere in the world. These platforms typically emphasize secure access protocols, intelligent scheduling systems, and seamless integration with diverse laboratory equipment.

The existing literature underscores the critical importance of usability, scalability, and security in the design of remote laboratory systems. Key challenges identified include ensuring real-time interaction capabilities, maintaining hardware integration reliability, and providing adequate user support and training.

Several systems currently offer functionalities similar to those proposed in the Remote Lab project. The ISA represented an early successful implementation but is no longer operational and unavailable for public access. Similarly, LabShare [3], another notable platform, is currently inactive. Contemporary systems such as WebLab-Deusto [4] focus on specialized domains like electronics and instrumentation, providing tailored interfaces and tools for remote experimentation in specific fields.

Other significant contributions to the field include the Remote Laboratory Management System (RLMS) [5] and the Labshare Sahara framework [6], which have influenced modern approaches to remote laboratory architecture and user experience design. These systems serve as valuable references for the development of the Remote Lab platform, informing critical decisions related to system architecture, user experience optimization, and hardware integration strategies.

1.3 Document Structure

This report is organized as follows: Chapter 2 presents and describes the proposed system architecture, including detailed specifications and core functionalities. Building upon the understanding of system components, the database design, web API implementation, and web application development are described in Chapters 3, 4, and 5, respectively. Chapter 6 details the project organization methodology and deployment strategies employed. Chapter 7 presents

comprehensive testing procedures and validation results for the implemented system. Finally, Chapter 8 provides conclusions regarding the system's performance and outlines potential directions for future development and enhancement.

Chapter 2

Proposed Architecture

With the objective of implementing a platform to provide remote laboratory access, Figure 2.1 presents a high-level architecture of the proposed system. A user can remotely access the platform through a web interface. The server hosting the platform communicates with an external authentication service to verify user credentials and establish secure sessions. Once authenticated, users can remotely manipulate laboratory hardware through the platform's interface.

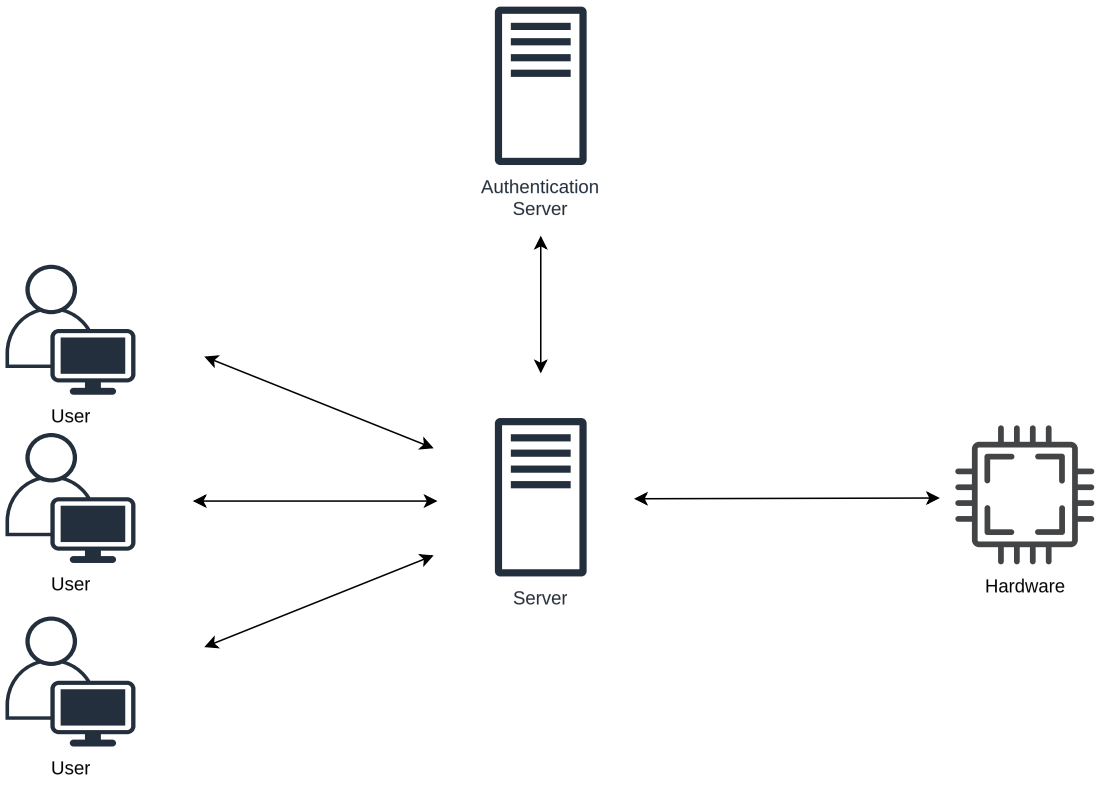


Figure 2.1: High-level Architecture

This chapter is structured to provide a comprehensive understanding of the system's design. Section 2.1 describes the different types of users and their respective interactions with the system. Section 2.2 introduces the core system functionalities and their implementation

approach. Finally, Section 2.3 presents the detailed system architecture with an introduction to its individual components and the technologies employed.

2.1 System Users

To ensure proper access control and functionality separation, the system defines three user roles, each with specific permissions and capabilities:

i. **Student** can access assigned laboratories and manipulate their associated hardware within defined parameters and time constraints;

ii. **Teacher** can create and configure laboratories, assign hardware resources, create and manage student groups, and monitor laboratory usage and performance;

iii. **Administrator** can manage system users, configure system-wide settings, monitor platform performance, and handle administrative aspects such as user provisioning and system maintenance.

A hierarchical system was designed in the following structure: *Student* → *Teacher* → *Administrator*. The administrator role inherits from the other roles. Teacher from student, and student as the base role. A user can only have one role assigned, which means that this system is design as a Hierarchical Role-Based Access Control (RBAC) [7].

2.2 System Functionalities

The system is built around several core functionalities that work together to provide a comprehensive remote laboratory experience. The platform supports robust user authentication and authorization mechanisms, ensuring secure access to laboratory resources. These security features are integrated throughout a Web API, which exposes well-defined endpoints that provide access to resources, implement resource management capabilities, and enforce business logic rules.

The Web API follows Representational State Transfer (REST) [8] principles and exposes resources through standardized Uniform Resource Identifiers (URIs) [9]. It maintains bidirectional communication with a purpose-built database designed to meet the system's requirements for storing and managing all necessary information, including user data, laboratory configurations, and session logs.

A critical component of the system is the hardware abstraction layer, which communicates with the Web API to translate high-level user instructions into hardware-specific commands. This abstraction enables the platform to support diverse types of laboratory equipment while maintaining a consistent user interface.

The Web Application serves as the primary user interface, making HyperText Transfer Protocol (HTTP) [10] requests to the Web API to provide users with comprehensive functionality, including:

- Secure user authentication and profile management;
- Laboratory discovery, access, and real-time interaction capabilities;
- Group creation and collaborative workspace management;
- Hardware configuration and monitoring interfaces;
- Comprehensive user and system administration tools.

2.3 System Components

The proposed system comprises four main components: a **Web Application**, **Web API**, **Database**, and **Hardware Abstraction layer**. The server entity depicted in Figure 2.1 is expanded in Figure 2.2 to reveal its internal components: the Back-End, Database, and Front-End. The architecture also illustrates the external Authentication Server and a Embedded Client, demonstrating the system's integration capabilities. Figure 2.2 illustrates the detailed architecture of the proposed system along with the specific technologies considered during the design phase.

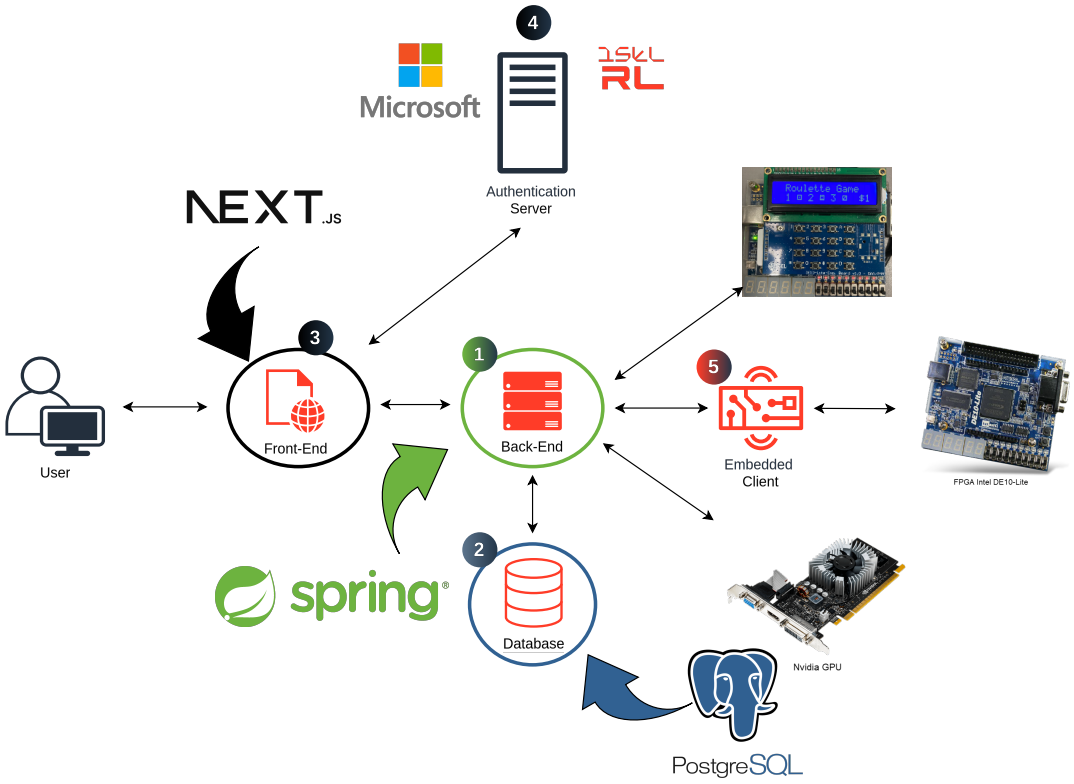


Figure 2.2: Detailed System Architecture

① The Back-End encompasses both the Web API and the Hardware Abstraction layer. The Web API uses HTTP as primary communication protocol and is built using Spring Framework [11] as the main technology stack, with Kotlin [12] as the primary programming language. Chapter 4 provides comprehensive details about the proposed Web API, including the rationale for choosing Spring [13] and Kotlin [12], architectural decisions, and implementation specifics;

② The Database is designed to efficiently store and manage all system information, including user profiles, laboratory configurations, hardware specifications, and groups information. PostgreSQL [14] was selected as the database management system. Chapter 3 describes the database entity-relationship model, explains the rationale behind choosing PostgreSQL [14], and provides detailed implementation information;

③ The Front-End is developed using Next.js [15], a modern React-based framework [16] that provides server-side rendering capabilities and excellent performance optimization features. It serves as the primary visual interface for users and handles all client-server communication by making HTTP requests to the Back-End for data retrieval and manipulation. Chapter 5 provides an in-depth analysis of the Web Application design and implementation;

④ User authentication is implemented through the Web Application using NextAuth.js [17], a comprehensive authentication framework that facilitates secure communication with external authentication providers. The Authentication Server is provided by Microsoft Open Authorization (OAuth) [18], enabling users to authenticate using their institutional credentials;

⑤ The Embedded Client is illustrated to demonstrate the system's hardware communication capabilities. In the proposed architecture, the Embedded Client communicates with Field-Programmable Gate Array (FPGA), showcasing the platform's ability to interface with programmable hardware devices. The hardware abstraction layer within the Back-End provides different instruction sets and interaction protocols depending on the specific hardware associated with each laboratory, ensuring compatibility with diverse equipment types while maintaining a consistent user experience.

Chapter 3

Database

The database serves as the foundational component of the system architecture. PostgreSQL [14] was selected as the database management system due to its open-source nature and robust support for relational data models. This choice aligns with previous project implementations and provides the consistency and performance required for the system's operational needs.

This chapter presents an overview of the Entity-Relationship Model (ER Model) and critical implementation details. Complete technical documentation is provided in the accompanying appendix A.

The database design follows a normalized relational structure that supports user authentication, secure session management, and the remaining system functionalities. The ER Model encompasses the core entities required for system functionality while maintaining data integrity and scalability.

3.0.1 Core Entities

The **USER** entity represents a user in the system, as depicted in figure 3.1. The *name* and *email* attributes are provided by the authentication system. The *role* serves as a discriminator attribute to identify whether the user is an Administrator, Teacher, or Student, as described in section 2.1

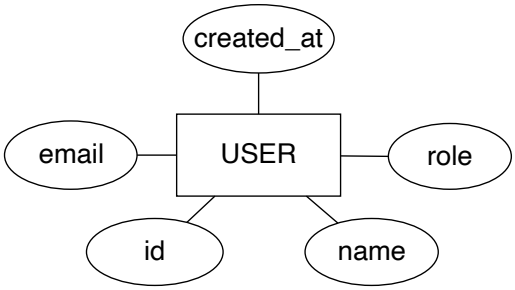


Figure 3.1: User Entity

A user can create N tokens. The **TOKEN**, as depicted in figure 3.2, is a weak entity because it cannot be uniquely identified by its attributes alone and therefore requires a user, which is a

strong entity, to be identified. A token, being unique, is created by only one user.

This is a useful entity for authentication purposes. It was designed to hold a hash value in the *token_validation* attribute.

Authentication workflow:

1. Upon successful user login, a unique token is created with cryptographically secure values and stored in the database.
2. For subsequent authenticated operations, the system queries the database to verify the client-provided token against stored values.
3. Valid tokens enable secure user identification without transmitting unique identifiers.

The *last_used_at* and the *created_at* are useful for determining token expiration. This expiration is determined in the application domain.

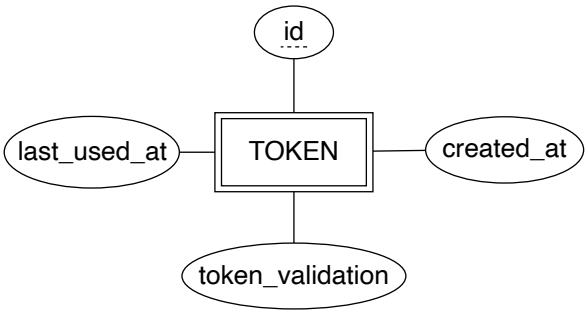


Figure 3.2: Token Entity

A user, as an Administrator or Teacher, can create N laboratories. When creating a **LABORATORY**, depicted in figure 3.3, the user can define the *name* and *description*. They can also define the *duration* of a laboratory session and its *queue_limit*.

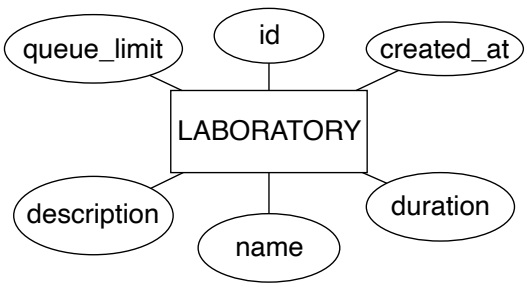


Figure 3.3: Laboratory Entity

Upon successful laboratory creation, the user can associate **HARDWARE**, depicted in figure 3.4 to it, which must be created separately.

For the creation, it requires a *name*, *Internet Protocol (IP) address* and *Media Access Control (MAC) address*, which can be null depending on the hardware, a *status* to indicate whether the

hardware is under maintenance, occupied, or available, and a serial number (*serial_num*) to uniquely identify the hardware. Although it has an identifier, the serial number helps physically identify the hardware.

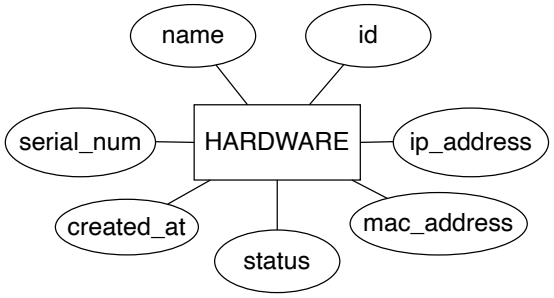


Figure 3.4: Hardware Entity

For a student to access a laboratory, they must be in a group that is associated with that laboratory. A professor can create a **GROUP**, depecited in figure 3.5 and associate users to it.

When creating a group, the user needs to *name* it and, optionally, add a *description* to it.

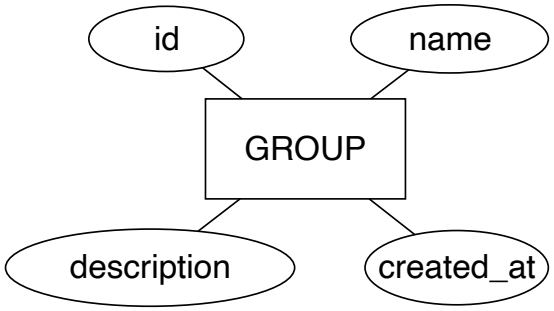


Figure 3.5: Group Entity

Besides the standard relationships between the entities described previously, there are two that require a detailed overview. The first is the *waiting_queue* relationship between the User and the Laboratory. Although it was established that a user must be part of a group associated with a laboratory to be able to join it, this constraint is not easily illustrated in a straightforward manner. Therefore, this relationship serves as a waiting queue for a specific laboratory.

The second relationship is the *session* relationship between the User, Laboratory, and Hardware. This is a crucial relationship for managing laboratory sessions and statistics. When a user successfully joins a laboratory, a session is created. Through this relationship, it becomes possible to control and track the sessions a user has within a laboratory.

3.0.2 Summary

Although PostgreSQL [14] is being utilized for its functionalities, it was decided that all logic and verifications would be implemented in the Web API, ensuring that no triggers or complex

constraints are implemented on the database side.

Figure 3.6 illustrates the ER Model, providing its entities and relationships.

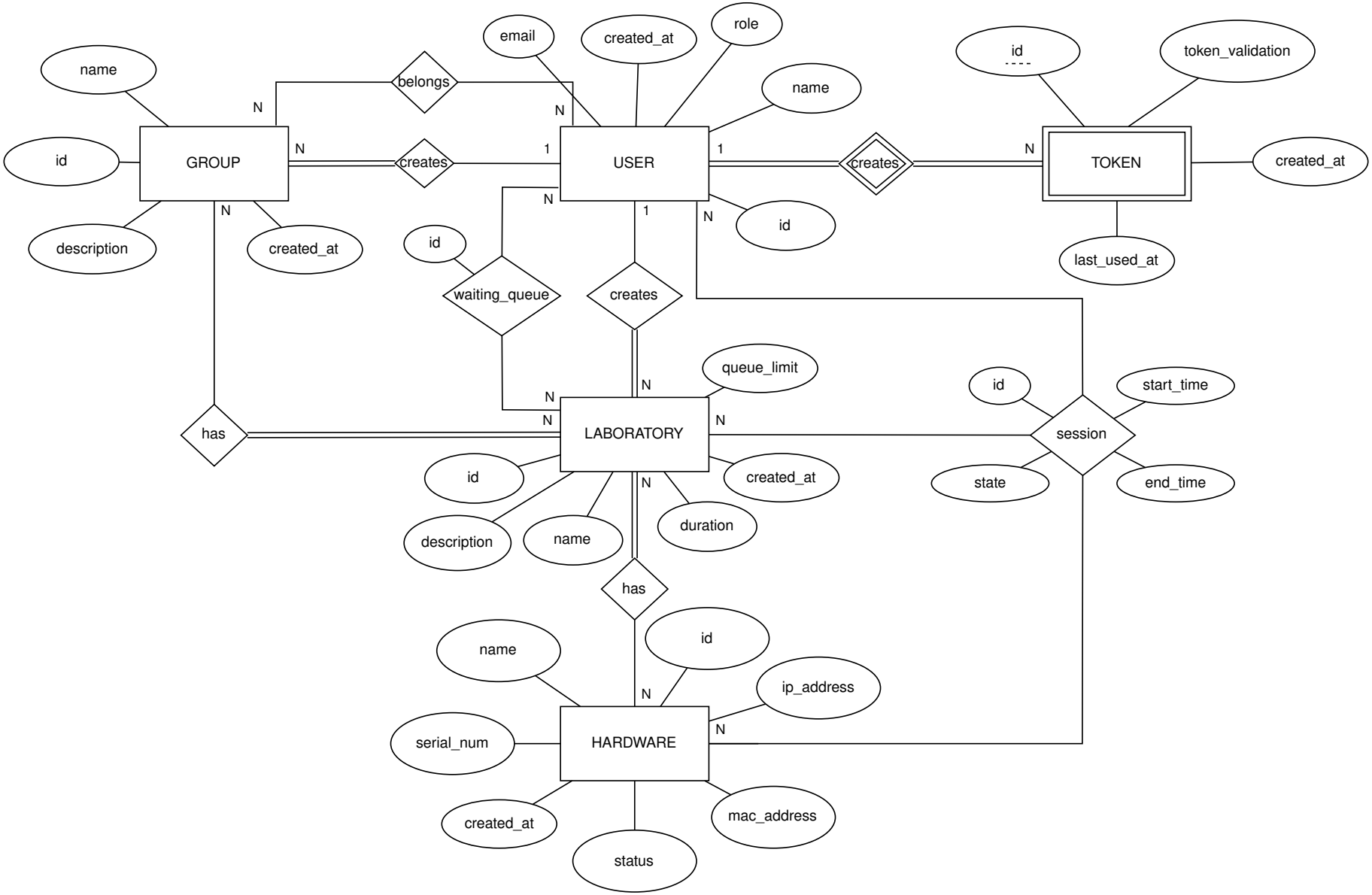


Figure 3.6: Entity-Relationship Model (ER Model)

Chapter 4

Web API

The Web API provides comprehensive endpoints for user management, authentication, authorization, and Create; Read; Update; Delete (CRUD) operations [19], serving as the backbone of the system's backend infrastructure.

It is developed using Kotlin [12] and the Spring Framework [11], following the Controller-Service-Repository pattern, which is prevalent in many Spring [13] applications. We chose this architectural pattern because of the clear separation of concerns it provides and the extensive testing possibilities it offers, enabling better maintainability and code quality.

To make the codebase even more maintainable and improve the developer experience during development, Spring Framework's Inversion of Control container [20] and the Strategy pattern [21] principle were also employed extensively throughout the application.

Spring's dependency injection [22] is a well-established technology in Java [23] enterprise programming. It provides an elegant way to declare dependencies, since the API was predominantly built following Object-Oriented Programming (OOP) principles. This framework allows us to declaratively specify the necessary dependencies for each module and provides a BeanFactory interface [24] for advanced configurations. Using these Spring technologies, object lifecycle management becomes Spring's responsibility.

The Strategy pattern allowed us to have greater control over specific implementations since it follows an interface-based approach. Every concrete implementation adheres to a well-defined interface, making it possible to change implementations dynamically without breaking existing functionality. This pattern is applied consistently across every module. For example, if it becomes necessary to change the repository implementation to use a different framework, it is only required to create a new class following the established interface, without breaking the existing codebase. Spring's dependency injection works exceptionally well with this strategy design pattern, making unit tests much more straightforward when the concrete implementation is not intended to be tested without modifying its code.

4.1 API Architecture

Figure 4.1 provides a comprehensive overview of the implemented API architecture. The **HTTP** module (Controller layer) is responsible for exposing the endpoints and handling incoming requests and responses. When a request is made, the **HTTP** module receives the request and passes it to the **Service** module. This is where the business logic and data validations are performed. Since it is necessary to fetch and persist data, a **Repository** module is required. The repository module is responsible for abstracting and managing all communication with the database layer.

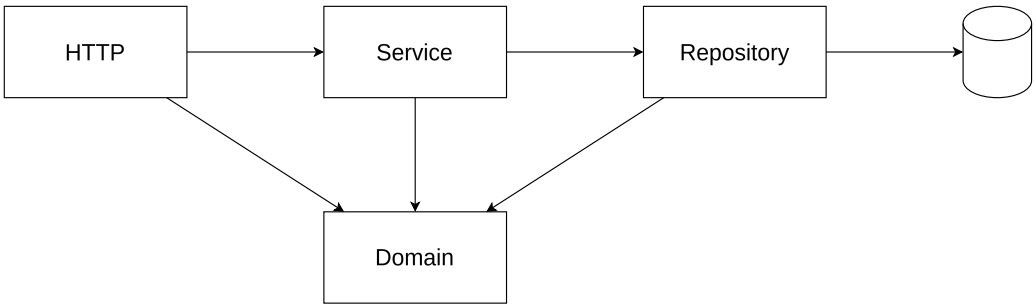


Figure 4.1: API Architecture Overview

This three-tier architecture promotes separation of concerns, making the system more modular, testable, and maintainable. Each layer has a specific responsibility and communicates only with adjacent layers, reducing coupling and improving overall system design.

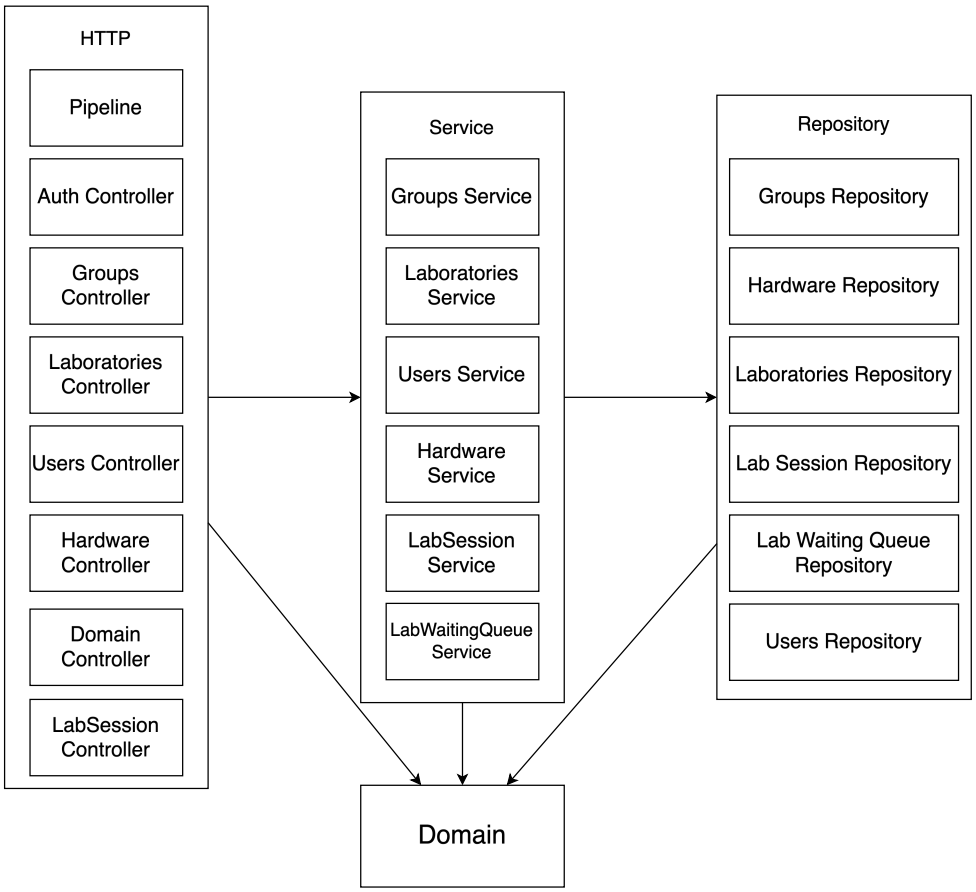


Figure 4.2: API Detailed Architecture

HTTP Module

The HTTP module contains the controllers [25], each with their respective functions and the request processing pipeline. The controllers are responsible for handling incoming requests, extracting and validating information, converting them into domain objects, and passing them to the next layer (Service Module). This layer also handles response formatting and HTTP status code management.

The pipeline contains filters [26] and interceptors [27] that implement cross-cutting concerns such as authentication, authorization, and logging. As the figure 4.3 depictes, filters intercept requests before they reach the *DispatcherServlet* [28]. They are part of the web server infrastructure and not the Spring framework itself. The interceptors are part of the Spring framework and operate between the *DispatcherServlet* and the controllers, providing more sophisticated request processing capabilities.

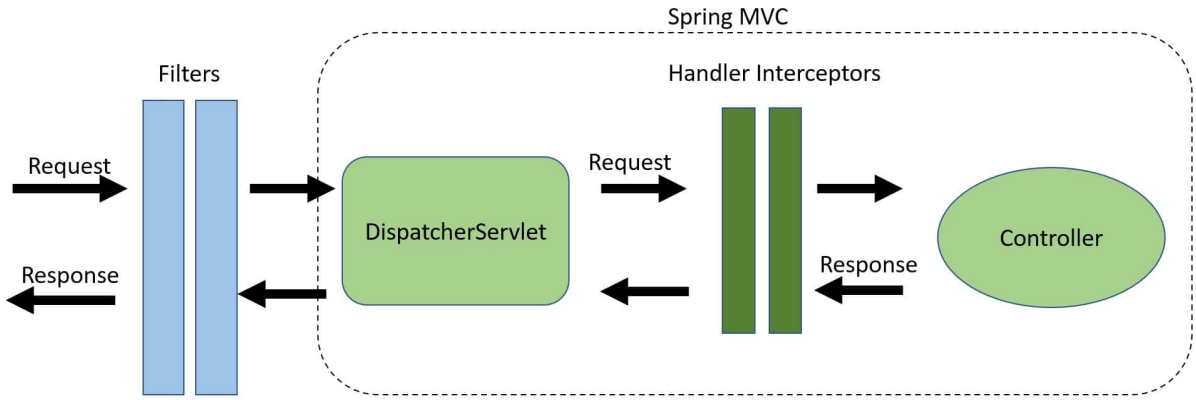


Figure 4.3: Comparison between filters and interceptors. Source: Baeldung [1].

Three custom interceptors were implemented: one to validate the API key, another to authenticate the user, and a third to verify the user role after authentication. This layered approach ensures that security concerns are handled systematically and consistently across all endpoints.

Since there are API endpoints that require an API key (for example, the login endpoint), a custom annotation was created to simplify this validation process. Every endpoint that needs an API key is annotated with *@RequireApiKey*. The interceptor responsible for this validation first verifies if the handler contains the annotation and, if present, verifies that the request contains the header *X-Api-Key* and validates its value against the configured API key. If the values do not match, an unauthorized error response is sent immediately.

The authentication interceptor checks if the handler has a parameter of type *AuthenticatedUser*, which encapsulates a token value and user information. If it does, it extracts the token from the request via header or cookie and validates it by retrieving the user associated with that token from the database. The user information is then added to the request as an attribute with a predetermined name, acting as a key for later retrieval. An argument resolver was implemented to resolve method parameters into argument values at runtime. In simple terms, for every handler that contains a parameter of type *AuthenticatedUser*, the parameter is resolved at runtime with the respective value. This resolution is accomplished by retrieving the attribute by the designated name and returning the corresponding value. This value should be the user information previously inserted by the authentication interceptor. If an error occurs during any of these steps, an unauthorized error response is sent.

The final interceptor implemented handles handlers that require a specific user role. This authorization mechanism is explained in more detail in Section 4.2.

The implemented filter is responsible for comprehensive request and response logging, providing valuable debugging information and audit trails for API usage.

These tools provided by the Spring Framework are invaluable for optimizing and simplifying cross-cutting concerns while maintaining clean separation of business logic.

Service Module

The service module performs the necessary business logic validations using domain classes defined in the Domain module. These classes provide configurations and methods for validating specific data types and business rules. Configurations in domain classes are provided by a JavaScript Object Notation (JSON) [29] file containing domain restrictions and validation rules. Listing 4.1 shows an example of the group domain and it's configurations.

```
1    "group": {
2        "groupName": {
3            "min": 3,
4            "max": 100,
5            "optional": false
6        },
7        "groupDescription": {
8            "min": 10,
9            "max": 1000,
10           "optional": true
11       }
12   }
```

Listing 4.1: Group Entry System Configuration Example

This JSON configuration file, provided in the appendix B, is converted to strongly-typed classes using Kotlin Serialization [30]. This approach allows for easy modification of specific validation rules without requiring codebase changes, promoting configuration-driven development and reducing deployment complexity.

The service layer also implements business logic such as user group management, laboratory session coordination, and complex validation rules that span multiple domain entities. This centralization of business logic ensures consistency and makes the system easier to maintain and extend.

Repository Module

The repository module serves as the data access layer, responsible for managing all communication between the application and the database. This functionality is implemented using Java Database Interface (JDBI) [31], a lightweight and efficient database access library that provides a clean, declarative API for database operations.

JDBI offers several advantages for database interaction through its declarative approach. It enables the use of interfaces, annotations, and custom mappers to perform database access operations without requiring extensive boilerplate code. The library is built on top of Java Database Connectivity (JDBC) [32], leveraging the standard Java database connectivity framework while providing a more intuitive and developer-friendly interface. This design philosophy results in a natural programming idiom that integrates seamlessly with both Java and Kotlin codebases.

The repository module implements the Repository pattern, which provides a consistent interface for data access operations while abstracting the underlying database implementation details. This abstraction allows for easy testing through mock implementations and provides flexibility for future database technology changes.

Additionally, the repository layer implements connection pooling, transaction management, and query optimization strategies to ensure optimal database performance under varying load conditions.

4.2 Roles

As mention, in Chapter 1 and Chapter 3, the user types in the system that supports the interactions and roles, a robust approach is needed to control these interactions to ensure proper access control to resources and maintain system security.

As mentioned previously, an user, when authenticated, is assigned a role, such as student, teacher, or administrator, which determines their permissions within the platform. To perform the necessary verifications, a custom annotation was created to be used in the controllers of the HTTP layer.

This annotation receives a role specification in its constructor. For the annotation to function properly, an interceptor was implemented that first checks if the handler contains the annotation so that it can bypass the verification for public endpoints, then extracts the required role and performs the authorization validation. The interceptor must be called after the authentication interceptor to be able to retrieve the user from the request and then access their respective role information.

Since this is a hierarchical permission system, if a handler contains an annotation with the student role, every authenticated user can access it, but if a handler contains an annotation with the teacher role, only users with teacher or administrator roles can access that handler, since the administrator role inherits all permissions from the teacher role. This hierarchical approach simplifies role management.

The *Role* class provides a method to validate this hierarchy by comparing the actual role of the user with the expected role, implementing the necessary logic to handle role inheritance and permission elevation.

This declarative approach to authorization makes the code more readable and maintainable while ensuring consistent security policy enforcement across all endpoints.

4.3 Laboratory Sessions

The laboratory management system allows users belonging to groups associated with specific laboratories to join and access remote terminals. A laboratory can have multiple hardware units associated with it, enabling a laboratory to accommodate multiple users simultaneously. This

distributed approach addresses two critical concerns: what happens when a laboratory has no available hardware for a user attempting to join, and how a session is managed in the backend infrastructure.

This section aims to clarify how these two concerns are addressed through queuing and session management mechanisms.

4.3.1 Waiting Queue

When describing the system architecture, it was mentioned that a laboratory implements a waiting queue mechanism. This is specifically designed for laboratories without immediately available hardware resources. When this situation occurs, the user enters a waiting queue that manages fair access to laboratory resources. The API maintains a stateless design, meaning that no persistent state is stored in memory, allowing multiple instances to run concurrently for improved scalability and reliability.

When a user attempts to enter a laboratory that has reached capacity, they are placed in a queue. A SseEmitter (Server-Sent Events) [33, 34] is returned to the user that sends real-time events when the queue position changes or when they successfully enter the laboratory. These events are initiated and managed by the server, providing a responsive user experience, as illustrated in figure 4.4.

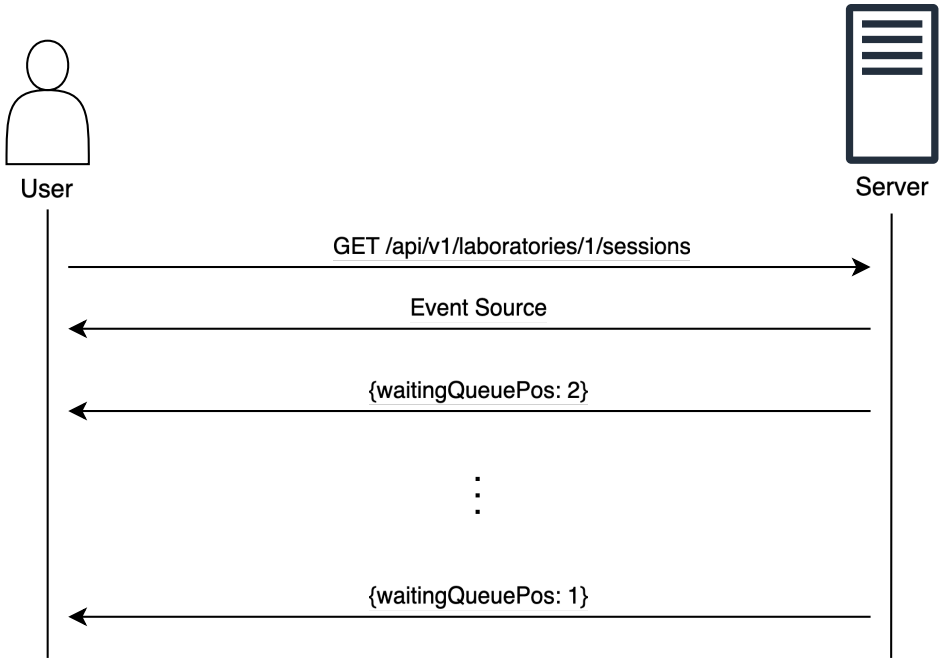


Figure 4.4: Server-Sent Events Example

An event source opens a persistent connection to an HTTP server, which sends events in *text/event-stream* format. The connection remains open until explicitly closed by calling *EventSource.close()* or when the session completes. This persistent connection model enables

real-time communication without the overhead of continuous polling.

Since the API maintains a stateless architecture, the queue state is persisted in the database and a Pub/Sub mechanism is used to notify users about changes in the queue position or when they can enter the laboratory. Kotlin coroutines [35] were also utilized to handle the server-side events efficiently, since the server takes the initiative in communication and needs to manage multiple concurrent connections.

For the Pub/Sub mechanism, Redis [36] was selected as the message broker. The Spring Framework provides comprehensive support for Redis, through Spring Data Redis [37], and can be deployed as a separate service, allowing multiple API instances to connect to it and share real-time state information seamlessly.

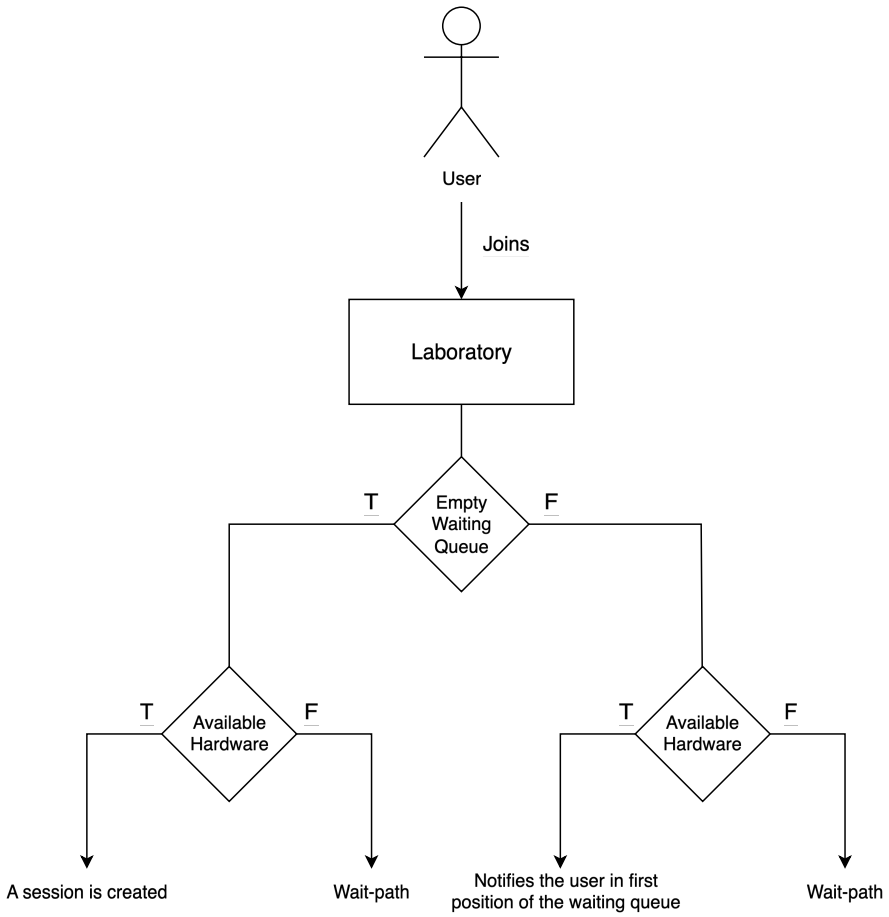


Figure 4.5: Laboratory Waiting Queue Scheme

The complete workflow of the waiting queue operation, as depicted in Figure 4.5, demonstrates that when a user attempts to join a laboratory, there are two possible execution paths: they can either immediately join the laboratory or be placed in the waiting queue.

When the laboratory's waiting queue is empty, a query is executed to retrieve the available hardware associated with that laboratory. The fast-path occurs when hardware is available, allowing the user to immediately join the laboratory and begin their session. The wait-path

occurs when no hardware is available. In this case, a database query is executed to insert the user ID into the waiting queue to maintain state, and a coroutine is created. Kotlin coroutines provide an optimal mechanism for executing code in parallel. Within the coroutine, the user subscribes to a channel through the Pub/Sub service (Redis), using the format *"lab:labId:queue:userId"*, where *labId* is the laboratory identifier and *userId* is the identifier of the user attempting to join. Redis supports subscribing to channels that do not yet exist, creating them at the moment of subscription. A suspension point is established, waiting for a message in the subscribed channel. This specific channel format is crucial for the notification phase and ensures message routing accuracy.

As Figure 4.6 illustrates, there are two completely separate execution contexts: the normal execution flow, where information extraction occurs from the request and is passed to the services, and the coroutine context. It is important to note that the SseEmitter creation occurs in the normal execution flow and not within the coroutine itself, but is provided to the coroutine. This allows the user to receive events produced by the coroutine.

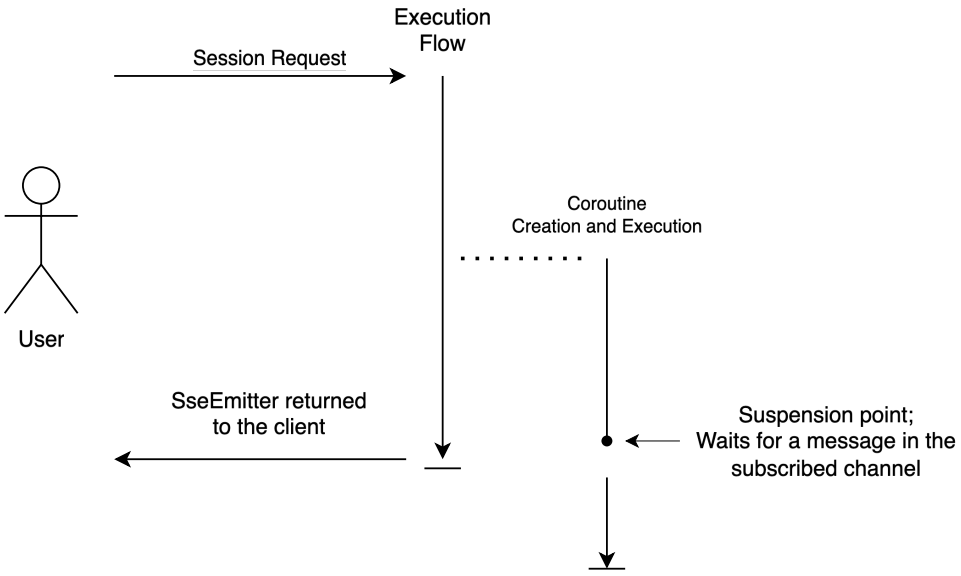


Figure 4.6: Wait-Path Scheme

Conversely, if the waiting queue is not empty, it indicates that the laboratory is likely at capacity and no hardware is available. To ensure accuracy, the system verifies whether hardware is available for the laboratory. This can occur if hardware was associated with the laboratory in the meantime, or if hardware that was under maintenance became available. The waiting queue is processed using FIFO (First-In-First-Out) logic, returning the ID of the user in the first position. With both the *lab ID* and the *user ID*, a message is published to the channel following the previously mentioned format. This is why the channel format must be strictly adhered to. The user waiting for a message receives it and processes its content. If the message is an empty string, a session is created immediately. If it contains the string *keepInQueue*, it means that

the user must remain in the queue and update their position by sending an event with the new queue position. This entire process is performed for every available hardware unit discovered, allowing multiple waiting users to join the laboratory. If no hardware is available, the user is placed in the queue and follows the wait-path mentioned previously.

When a session ends, verifications are performed to allow waiting users to enter the laboratory. This queuing mechanism ensures fair access to laboratory resources while maintaining system responsiveness and providing users with real-time feedback about their position and expected wait times.

4.3.2 Session

As previously mentioned, coroutines are utilized to send events through the SseEmitter and handle them efficiently. A laboratory session has a configured duration limit, which is communicated to users via events by sending periodic updates with the current state and remaining time.

A notification interval is configured, enabling the system to send events indicating how much time remains until the session ends. For example, if a laboratory has a duration of three hours, events can be sent every hour indicating the remaining time, such as sending a notification that two hours remain after one hour has passed from the start of the session. The system also implements a countdown mechanism, sending events when critical time thresholds are reached, such as when fifteen minutes remain. When the session has ended, an event is sent with type *session_finished*, allowing the client to handle session cleanup appropriately.

Using coroutines, these steps are simplified, allowing optimal performance on the backend side. If the user was in the queue, the coroutine created is used to manage the session by reusing the previously created resources.

Additionally, the session management system includes automatic session cleanup to free hardware resources and graceful session termination with proper resource deallocation.

4.4 Summary

The Web API implements several security measures to protect against common web application vulnerabilities, utilizing authentication, authorization, and input validation. It is also designed with performance and scalability in mind, providing a stateless design, database connection pooling, asynchronous processing, and message queuing. Finally, the system includes detailed logging of all API requests and responses.

It represents a robust, scalable, and secure backend solution that effectively addresses the complex requirements of laboratory management and user interaction. The architectural decisions made, including the use of the Controller-Service-Repository pattern, Spring Framework's dependency injection, and the Strategy pattern, have resulted in a maintainable and extensible

codebase.

Since the API is expected to be made public in future work, comprehensive documentation has been published on Postman [38]. The public API documentation [39] and the private API documentation [40] are hosted as publicly accessible websites on Postman. This provides an easy-to-use documentation builder with interactive examples and testing capabilities. When the API reaches a stable version and is made public, users will be able to generate authorization tokens via the web interface, enabling third-party integration and extending the system's utility beyond its initial scope.

The system's architecture provides a solid foundation for future enhancements, including additional laboratory types, advanced scheduling features, and integration with external educational platforms.

Chapter 5

Web Application

The web application is the primary interface for users to interact with the Remote Lab platform. Built with Next.js (React), it provides a modern, responsive, and user-friendly experience for students, professors, and administrators [15]. The application enables users to access, schedule, and manage laboratory resources remotely, supporting a wide range of devices and ensuring accessibility for all user roles.

The user interface is built using Ant Design [41], a comprehensive UI design language and React component library that provides a rich set of pre-built components. This choice ensures consistency in design, rapid development, and a professional appearance across all application screens. Ant Design components are used throughout the application for forms, tables, modals, navigation elements, and other interactive components, providing a cohesive and intuitive user experience.

This chapter serves as the main documentation for the @Web (Next.js) application, providing a comprehensive overview of its architecture, features, authentication mechanisms, and integration strategies. It is intended as the primary reference for understanding the design and implementation choices made in the development of the web application, and should be consulted for any details regarding the frontend of the Remote Lab platform [15].

5.1 Overview

The web application serves as the main point of interaction, integrating seamlessly with backend services via RESTful APIs. It supports multiple user roles, each with tailored access and features according to their permissions. The interface dynamically adapts to the user's role, ensuring that each user only sees the features and data relevant to them.

The decision to use Next.js as the framework was driven by its robust feature set and alignment with modern web development best practices. Next.js, built on top of React, offers built-in solutions for routing, server-side rendering, API integration, and more [42, 43, 44, 45]. This allowed the team to leverage existing knowledge while accelerating development and ensuring scalability and maintainability.

A key architectural choice is the retrieval of domain configuration in JSON format from the backend API. This ensures that the frontend's domain-related settings are always synchronized with those defined on the backend, centralizing domain management and reducing the risk of inconsistencies.

5.2 Architecture and Logic Separation

The application leverages Next.js to achieve a clear separation between client-side and server-side logic. All sensitive operations and data exchanges with the backend API are performed server-side, enhancing security and control over data flow [43, 44]. This architecture enables efficient server-side rendering, improved performance, and a better user experience. For more details on using Next.js to perform API requests from the server, see [46].

To optimize performance and user experience, the frontend performs input validation before sending requests to the API. This approach reduces the number of unnecessary API calls by catching invalid data early, providing immediate feedback to users, and minimizing server load. Importantly, the validation rules on the frontend are kept synchronized with the backend by leveraging the domain configuration JSON retrieved from the API. This ensures that both the client and server enforce consistent validation logic, reducing the risk of discrepancies and improving the reliability of the application.

The application implements a dynamic form system where the required fields, validation rules, and component types are determined by the domain configuration retrieved from the backend API. This configuration-driven approach ensures that forms automatically adapt to changes in business rules without requiring frontend code modifications. The domain configuration specifies field restrictions such as minimum and maximum values, optional/required status, and appropriate input components (text areas, number inputs, select dropdowns, etc.). This architecture provides flexibility and maintainability, as all form behavior is centrally managed through the backend domain configuration.

The application implements a comprehensive notification system that provides real-time feedback to users during various operations. Notifications are displayed for both successful and failed operations, ensuring users are always informed about the status of their actions. This includes notifications for laboratory creation, group management, hardware configuration, user operations, and session management. The notification system enhances user experience by providing immediate feedback, reducing uncertainty, and helping users understand the outcome of their interactions with the platform.

5.3 Main Features

- **Authentication:** Secure login using Microsoft OAuth (NextAuth), supporting university credentials and automatic role assignment [17].

- **Dashboard:** Personalized dashboard displaying relevant information, upcoming sessions, and quick access to key features.
- **Laboratory Management:** Professors and administrators can create, edit, and manage laboratory sessions, equipment, and participant lists.
- **Role-Based Access:** The interface adapts to the user's role, showing only the features and data relevant to their permissions. Users with higher roles can view and interact with the platform as if they had a lower role for testing and support purposes (RBAC).
- **Responsive Design:** Optimized for desktops, tablets, and mobile devices, ensuring accessibility and usability across platforms.
- **Loading UI:** The application uses loading states and skeleton screens to provide feedback during data fetching, improving perceived performance [47].
- **Terminal Integration:** The application integrates xterm.js [48] for real-time terminal access to hardware devices. This terminal component establishes WebSocket [49] connections with the backend API, enabling direct communication with laboratory hardware. Users can interact with hardware devices through a full-featured terminal interface, providing a seamless remote laboratory experience with real-time command execution and response display.

An additional usability feature implemented in the frontend is the ability for users with higher roles to switch their view to that of a lower role. For example, an administrator can choose to view the application as a professor, and a professor can view it as a student. This functionality is particularly useful for testing, support, and understanding the user experience from different perspectives, ensuring that features and permissions are correctly enforced for each role.

This temporary role switching functionality is implemented using React Context API through a dedicated `TempRoleContext`. The context provides a centralized state management solution that allows any component in the application to access and modify the current temporary role. The implementation includes several key features:

- **Role Hierarchy Validation:** The context enforces role hierarchy rules, ensuring users can only switch to roles they have permission to access. Teachers can switch to student view, and administrators can switch to any role.
- **Persistence:** The temporary role is stored in the browser's localStorage, allowing the user's role preference to persist across browser sessions and page refreshes.
- **Cross-Tab Synchronization:** The context listens for localStorage changes across browser tabs, ensuring that role changes are synchronized when the user has multiple tabs open.

- **Automatic Fallback:** If a user's real role changes (e.g., through admin action), the temporary role automatically reverts to the user's actual role to maintain security.

- **Error Handling:** Invalid role assignments are prevented, and users are redirected to an error page if they attempt to access unauthorized roles.

The context is integrated into the application's root layout, wrapping all components and providing the temporary role state throughout the component tree. Components can access the current temporary role using the `useTempRole` hook, which returns both the current temporary role and a function to update it. This design ensures that the role switching functionality is available consistently across all application screens while maintaining proper security boundaries.

To enhance the user experience and make the application feel as much like a single-page application with multiple menus as possible, the frontend leverages Next.js modals using parallel and intercepting routes [42]. This approach allows users to open dialogs, forms, or detailed views as overlays without leaving the current page context, providing smooth navigation and reducing full page reloads.

The modal implementation is designed with flexibility and accessibility in mind. Modal information and configurations are stored in external files, which are then imported and used by Next.js to generate both modal overlays and complete standalone pages. This architecture enables users to access modal content directly through URIs, providing complete pages when accessed directly while maintaining the modal overlay experience when navigated to from within the application.

Additionally, the modals feature enhanced user interaction capabilities, including the ability to resize and move them around the screen. This resizable and movable functionality allows users to customize their workspace layout, making it easier to work with multiple modals simultaneously or to position them according to their preferences. These features contribute to a more flexible and user-friendly interface that adapts to different user workflows and screen configurations.

The application follows a philosophy of enabling users to perform all operations within the same page context while maintaining the flexibility to access different pages directly through URI input. This approach is supported by a comprehensive breadcrumb navigation system that provides clear context about the user's current location within the application hierarchy. The breadcrumb component displays the navigation path and allows users to quickly navigate back to previous levels, enhancing the overall navigation experience while maintaining the single-page application feel.

By utilizing these advanced routing features and interactive capabilities, the application maintains a cohesive and interactive interface, improving usability and workflow efficiency.

5.4 Authentication with NextAuth

Authentication is implemented using NextAuth.js [50], a flexible authentication solution for Next.js applications. NextAuth is integrated to provide secure, seamless login experiences using Microsoft OAuth, allowing users to authenticate with their university credentials [17].

When a user attempts to log in, they are redirected to the Microsoft login page. Upon successful authentication, NextAuth handles the OAuth flow, retrieves the user's profile information, and establishes a session.

NextAuth manages user sessions securely, storing authentication tokens and session data in HTTP-only cookies to prevent unauthorized access from the client side. The authentication state is accessible throughout the application, enabling role-based access control and dynamic adaptation of the user interface according to the user's permissions.

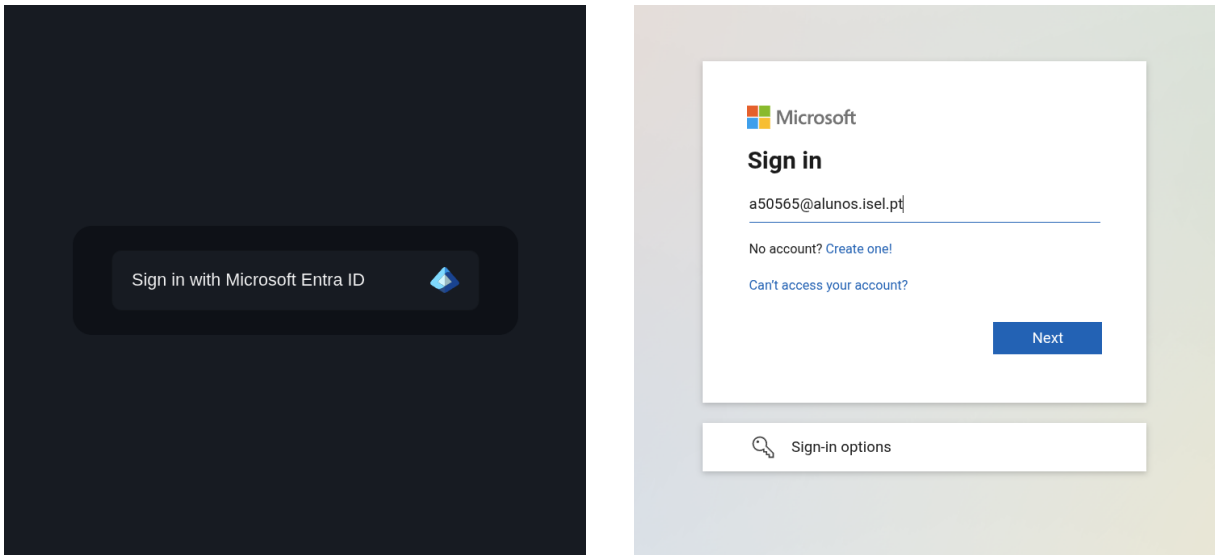
This approach ensures that only authorized users can access protected resources and features, while also providing a familiar and convenient login experience. The integration with Microsoft OAuth leverages the institution's existing identity infrastructure, enhancing security and simplifying user management.

NextAuth.js [50] is highly extensible and supports a wide range of authentication providers through its flexible OAuth configuration [17]. This means that, in addition to Microsoft OAuth, it is possible to integrate other OAuth-based authentication methods commonly used by universities, such as Google, GitHub, or institutional identity providers that support OAuth 2.0 or OpenID Connect. Furthermore, NextAuth allows the implementation of custom OAuth providers, making it feasible to connect to a university's own authentication server if needed. This flexibility ensures that the authentication system can adapt to different institutional requirements and future changes in identity management strategies [17].

5.5 User Interface

This section presents a comprehensive overview of the web application's user interface through screenshots that demonstrate the various features and user flows available to different roles within the platform.

Authentication is enforced throughout the application. When the user clicks the login button on the login screen (Figure 5.1a), they are redirected to the Microsoft OAuth login page (Figure 5.1b). Upon successful authentication, the user lands on the initial page of the Remote Lab platform, which provides access to the main features (Figure **??**).



(a) Login screen

(b) Microsoft OAuth login (after clicking the login button)

Figure 5.1: Authentication flow: (a) login screen; (b) Microsoft OAuth login page shown after clicking the login button.

The interface dynamically adapts to the user's role. Users with higher permissions can switch their view to lower roles for testing and support purposes (Figure 5.2). For example, the professor dashboard displays the main interface and available features for laboratory management (Figure 5.3).

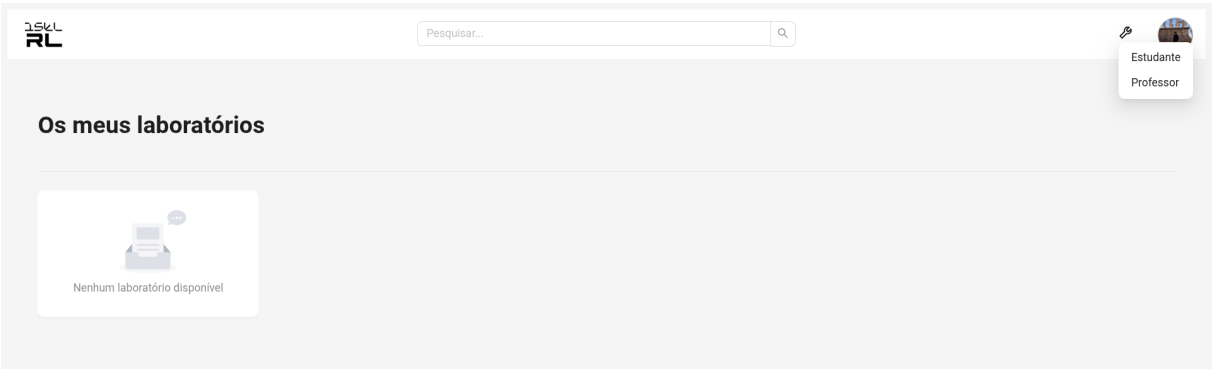


Figure 5.2: Role selection dropdown

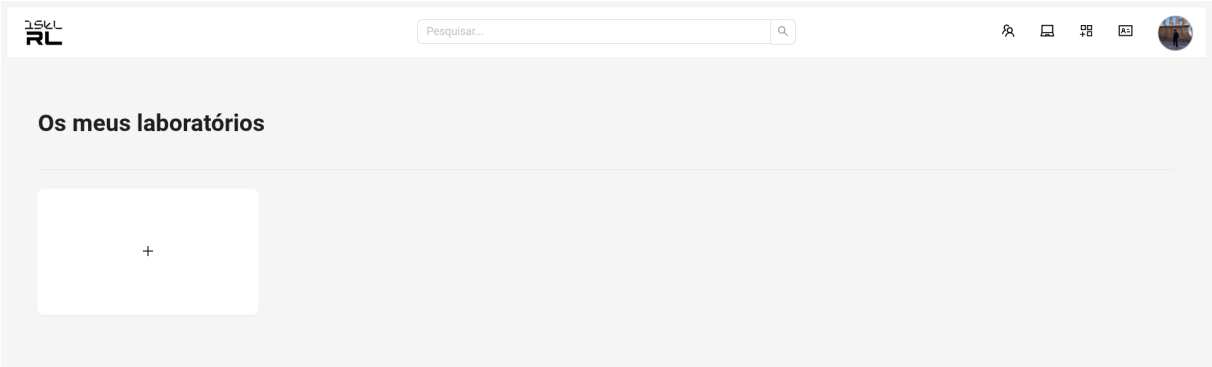


Figure 5.3: Professor dashboard

Laboratory creation and management are performed through interactive modals, which include real-time input validation and provide a smooth user experience. These modals are both resizable and movable, allowing users to customize their workspace layout. The application is designed to behave as a single-page application, with most operations performed in overlays. However, if a user accesses the URI of a modal directly, the same content is displayed as a full-page view, ensuring that all features remain accessible and consistent regardless of navigation method (Figure 5.4).

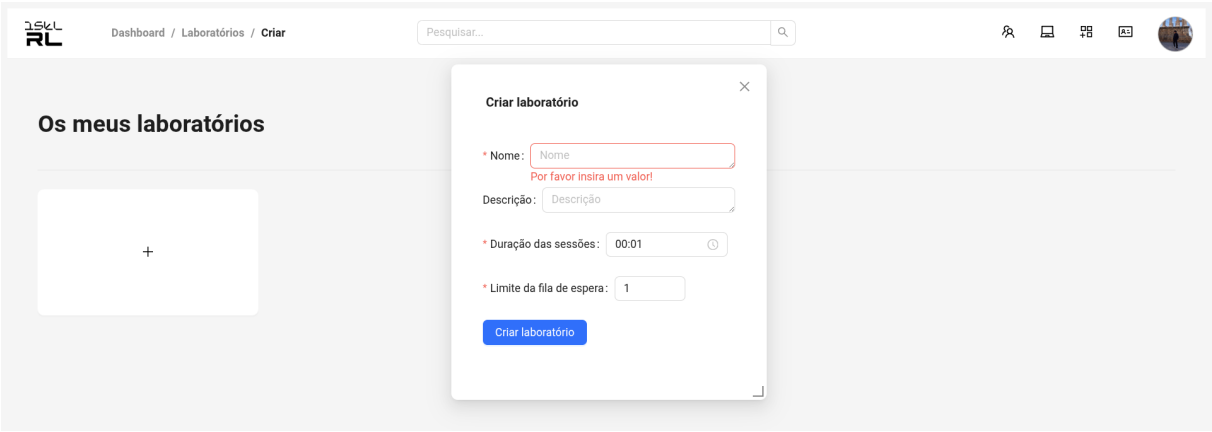


Figure 5.4: Lab creation modal

The flexibility of the modal system is illustrated in Figure 5.5, where the laboratory creation modal has been resized and moved, and input validation is demonstrated. Additionally, Figure 5.6 shows the same laboratory creation interface displayed in full screen, as it appears when the URI is accessed directly.

The modal form fields, including properties such as whether a field is required or optional, minimum and maximum values, and other validation rules, are dynamically retrieved from the backend API. This ensures that the frontend validation logic is always consistent with the business rules defined on the server, as described in Section 5.2. The configuration-driven approach allows the modal to automatically adapt to changes in the domain model without

requiring frontend code modifications. Validation is performed both on the website (client-side) and on the backend, reducing unnecessary API requests and providing immediate feedback to users.

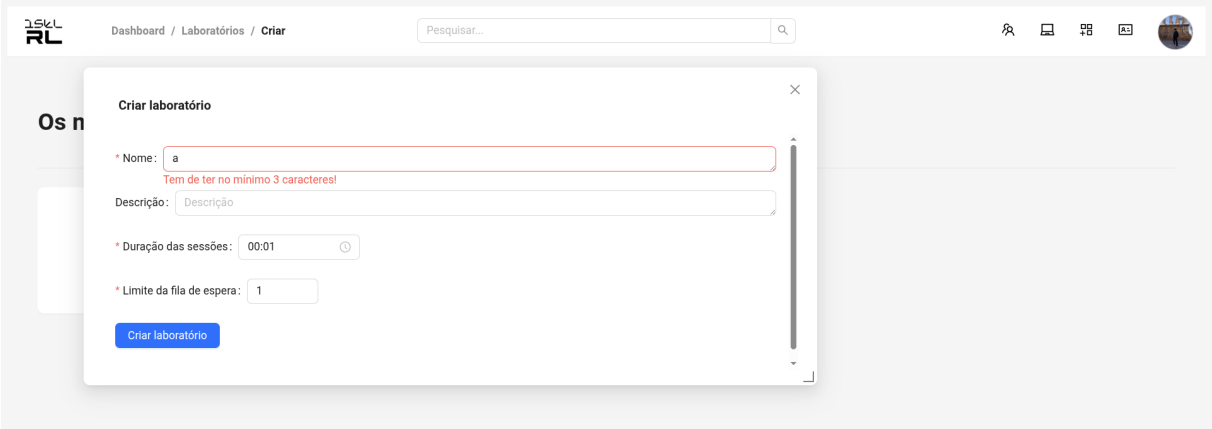


Figure 5.5: Interactive modal with validation

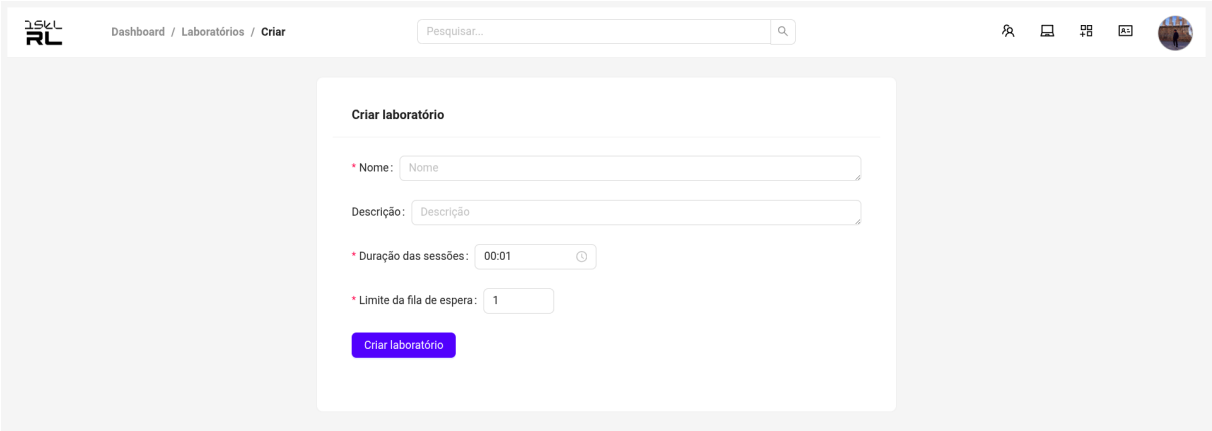


Figure 5.6: Full-screen lab creation

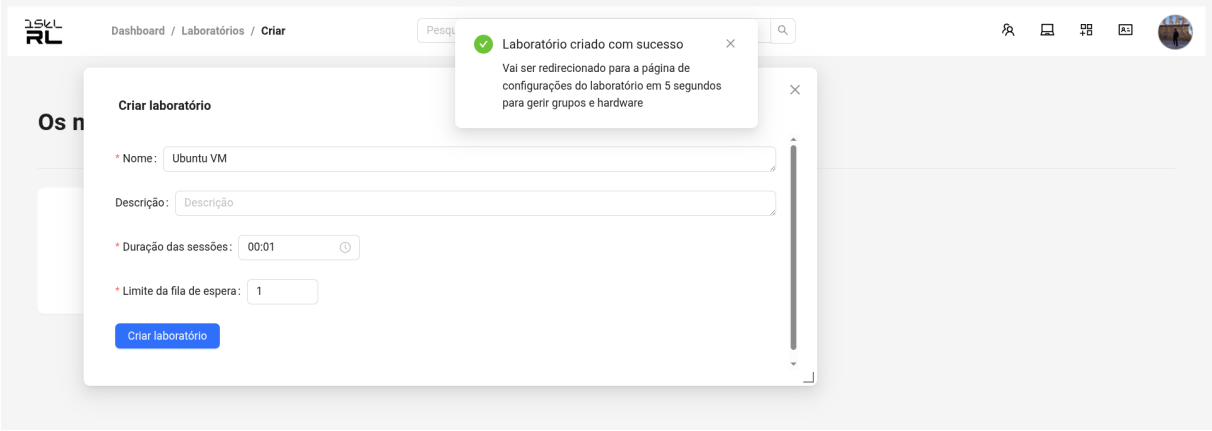


Figure 5.7: Success message after creation

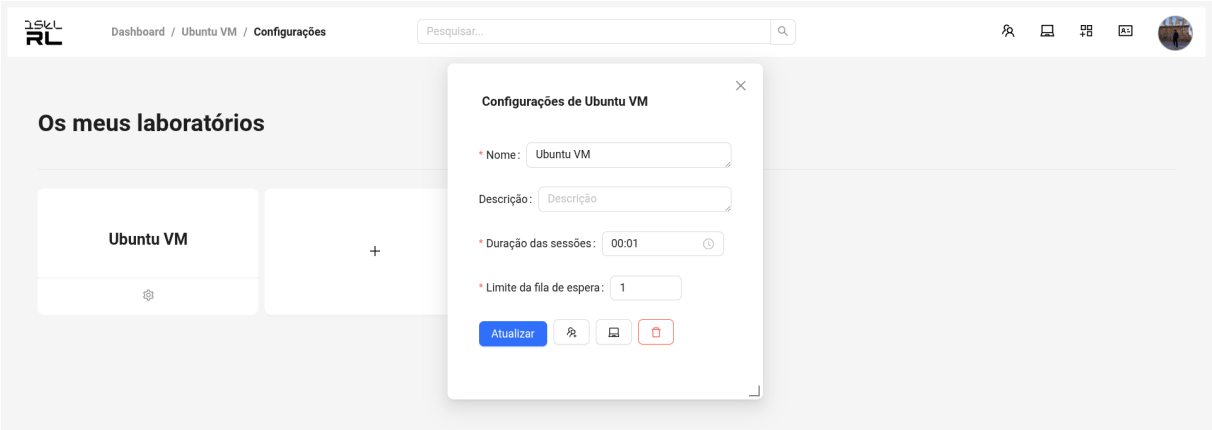


Figure 5.8: Lab editing interface

Group management can be performed either within a laboratory context or through the general management interface. The interface provides options for managing groups (Figure 5.9), creating new groups (Figure 5.10), confirming group creation (Figure 5.11), and managing group members (Figure 5.12).

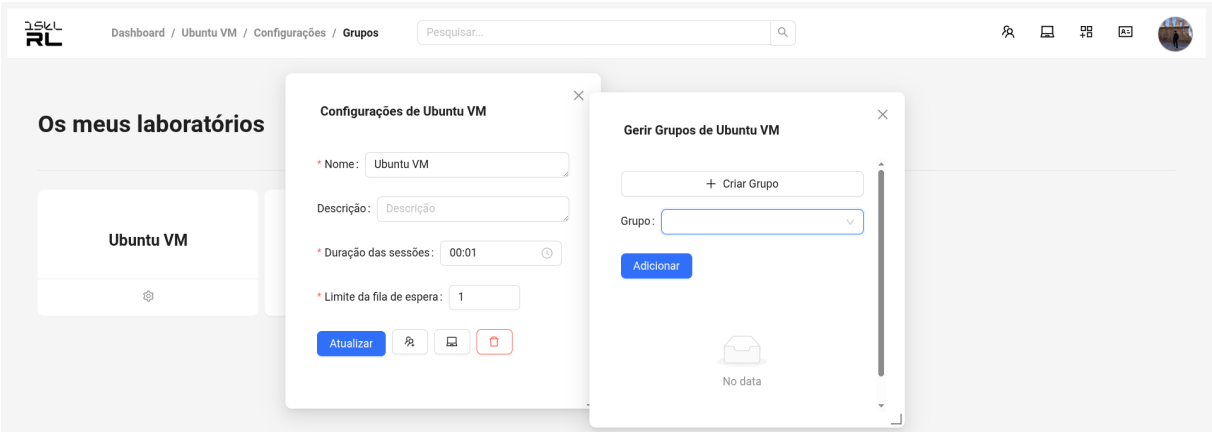


Figure 5.9: Lab group management

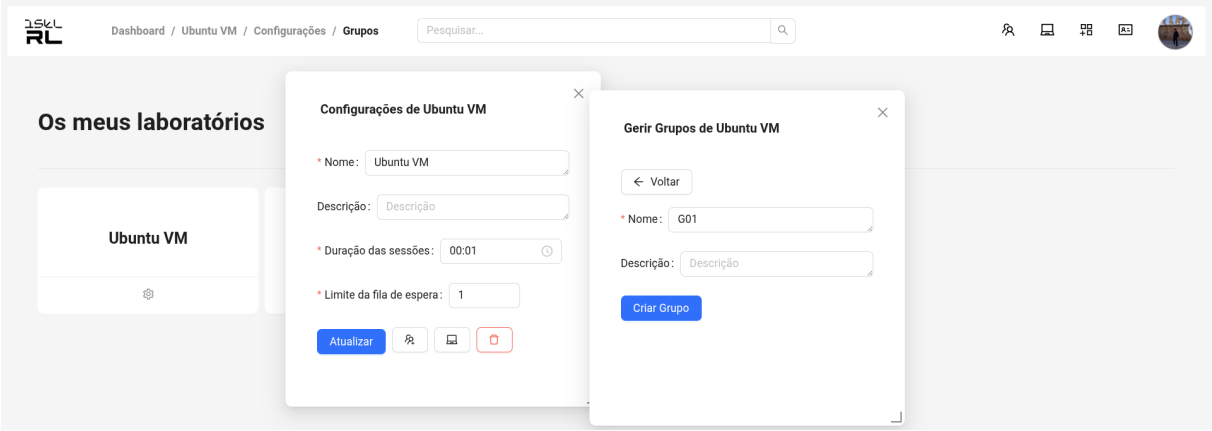


Figure 5.10: Group creation form

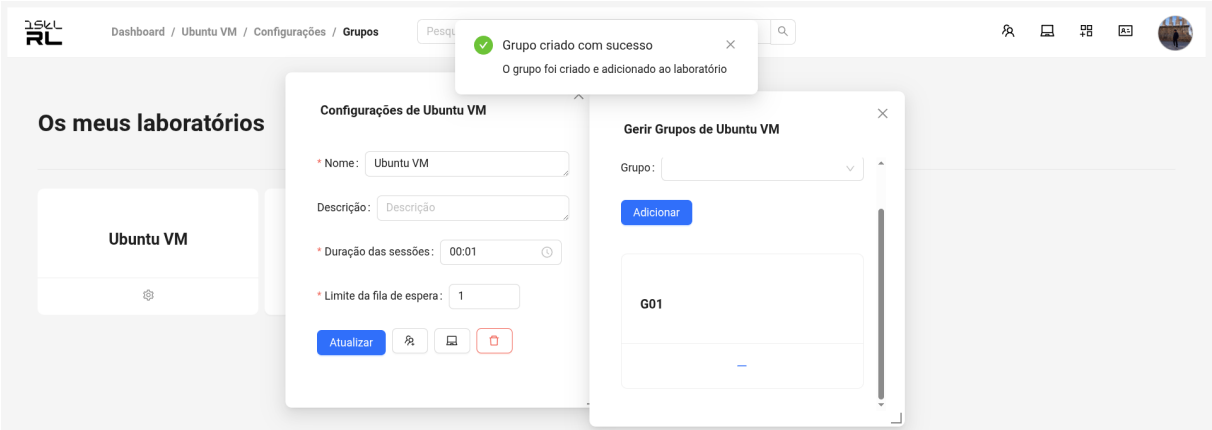


Figure 5.11: Group created confirmation

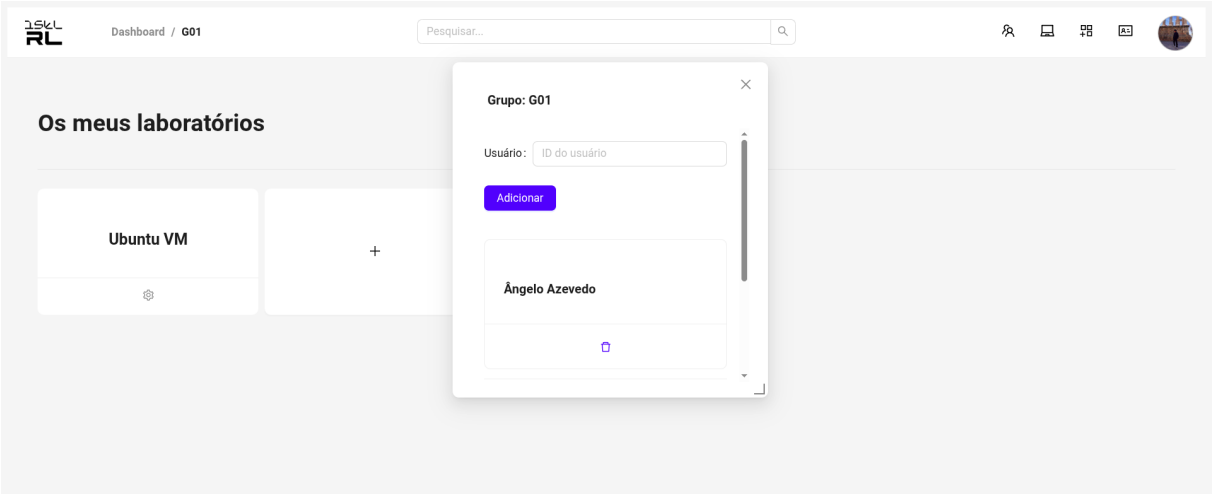


Figure 5.12: Group user management

Hardware management is available both within specific laboratory contexts and through a general management interface. Users can configure available equipment (Figure 5.13), add hardware to labs (Figure 5.14), and edit equipment settings (Figure 5.15).

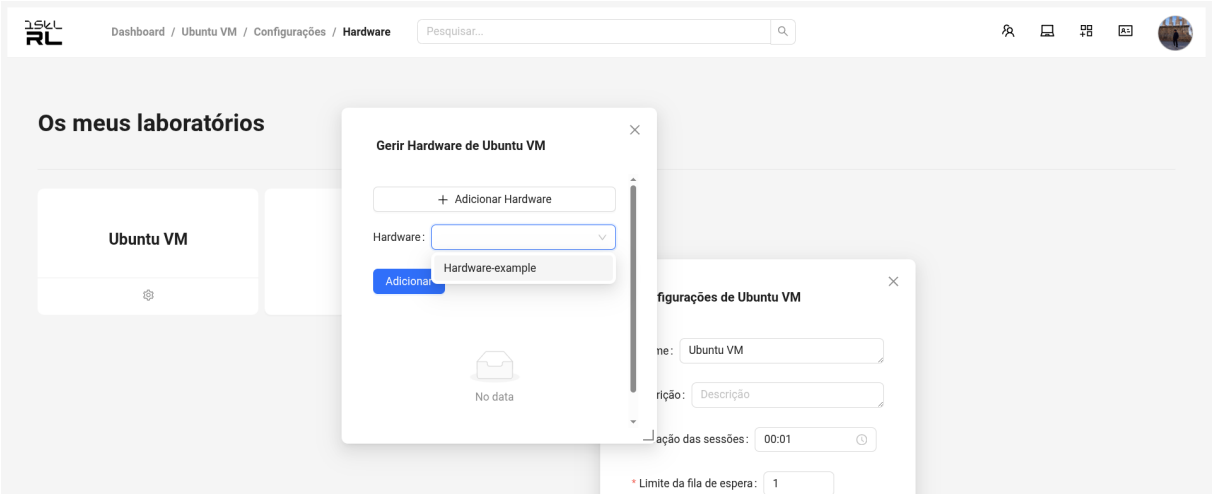


Figure 5.13: Lab hardware management

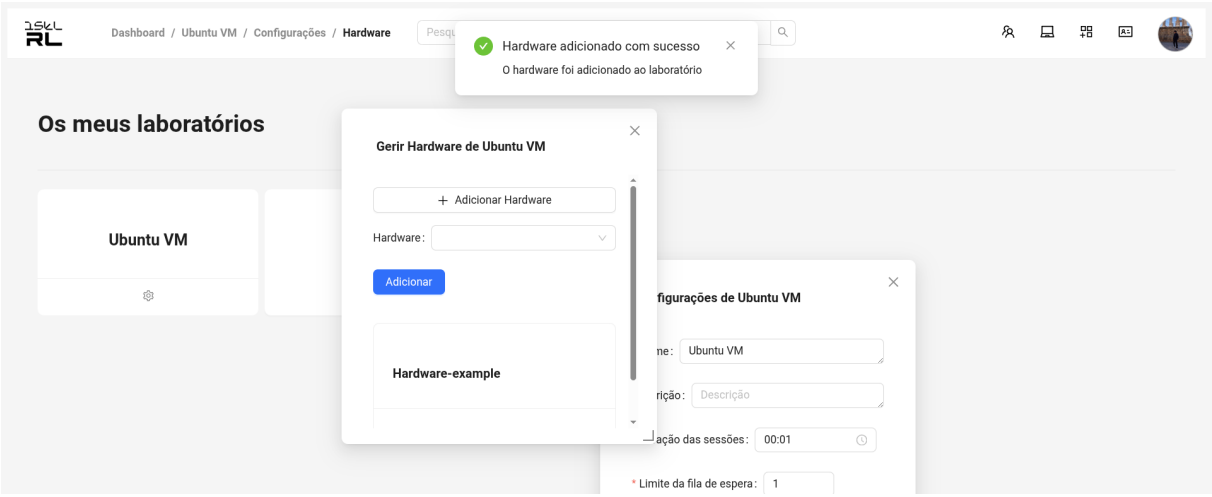


Figure 5.14: Hardware added to lab

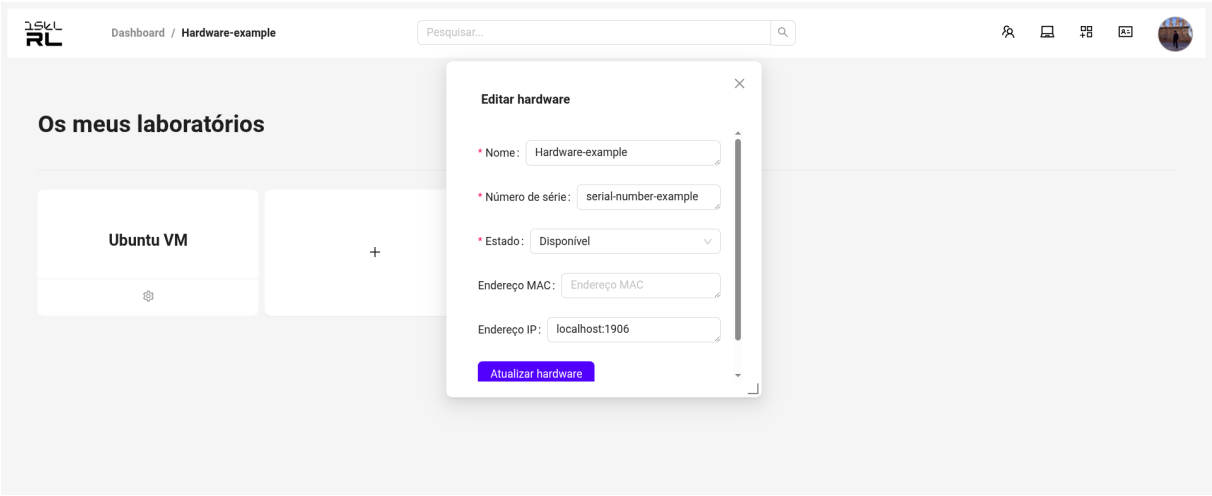


Figure 5.15: Hardware editing interface

During laboratory sessions, users can start a remote session (Figure 5.16) and, if necessary, wait in a queue until they gain access to the lab (Figure 5.17).

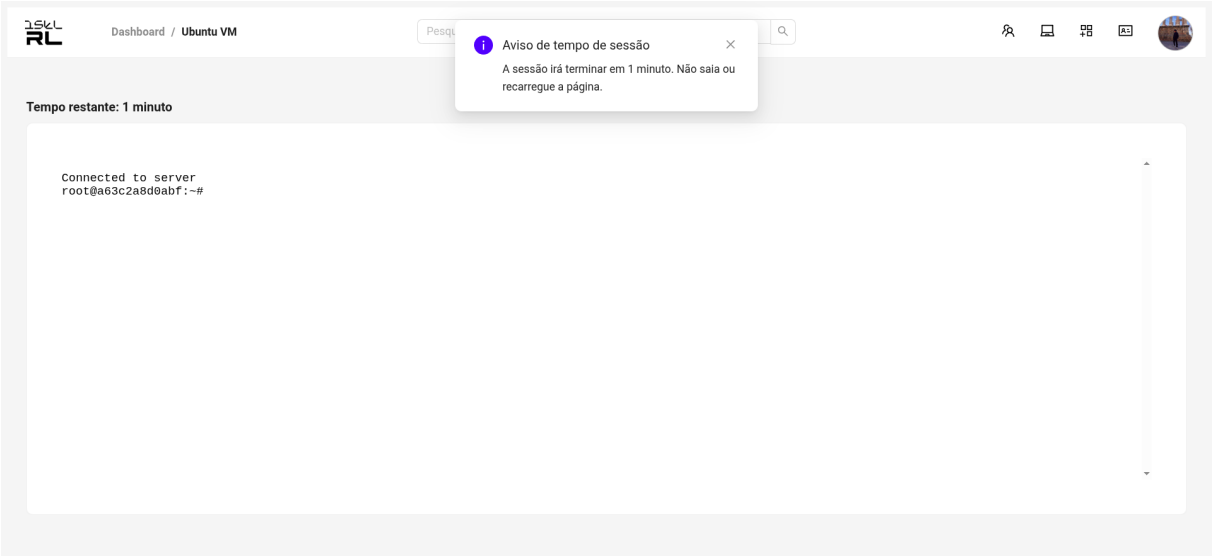


Figure 5.16: Lab session start

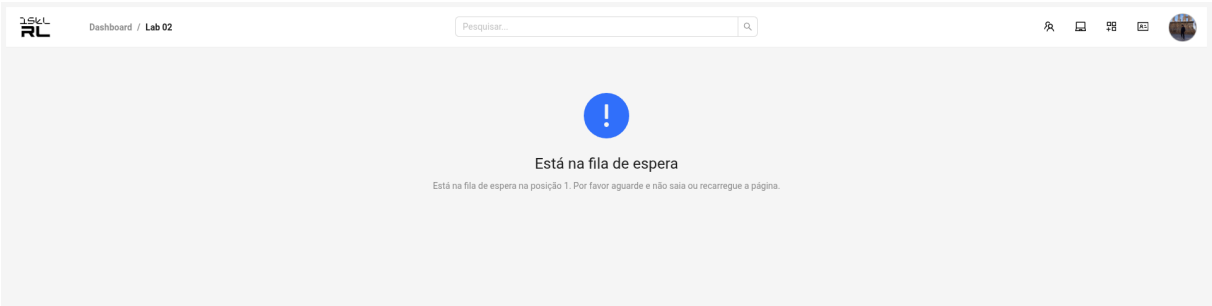


Figure 5.17: Waiting queue

The user account area provides a quick access menu to the profile, settings, and logout (Figure 5.18), as well as a detailed account page (Figure 5.19). The dropdown menu allows users to quickly navigate to their account details, shown below.

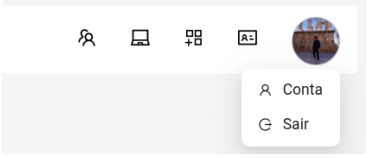


Figure 5.18: Account dropdown menu for quick access to profile, settings, and logout.

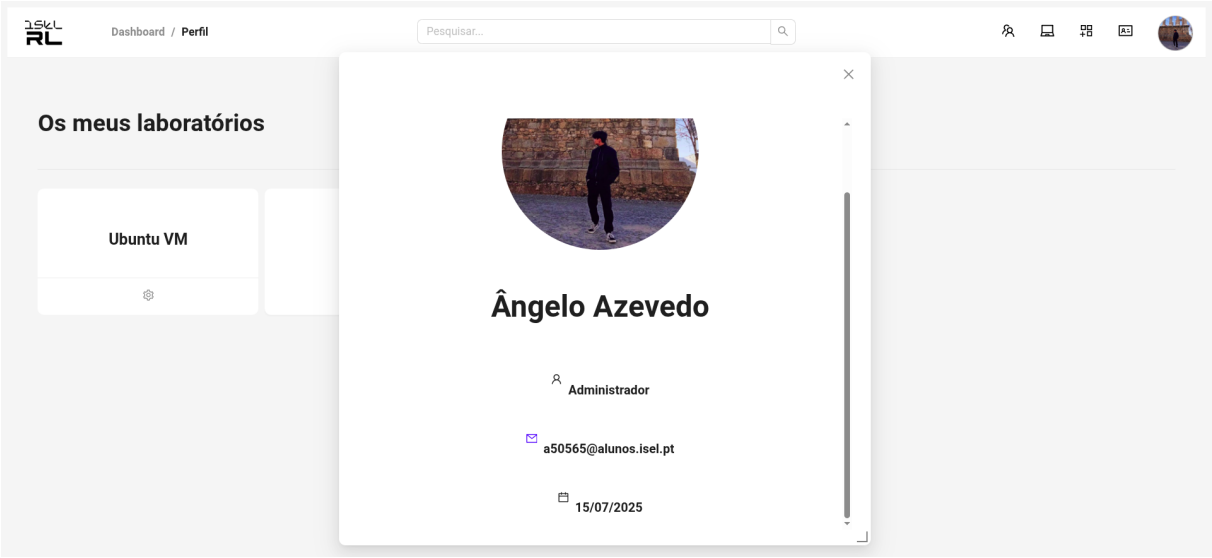


Figure 5.19: Detailed account information.

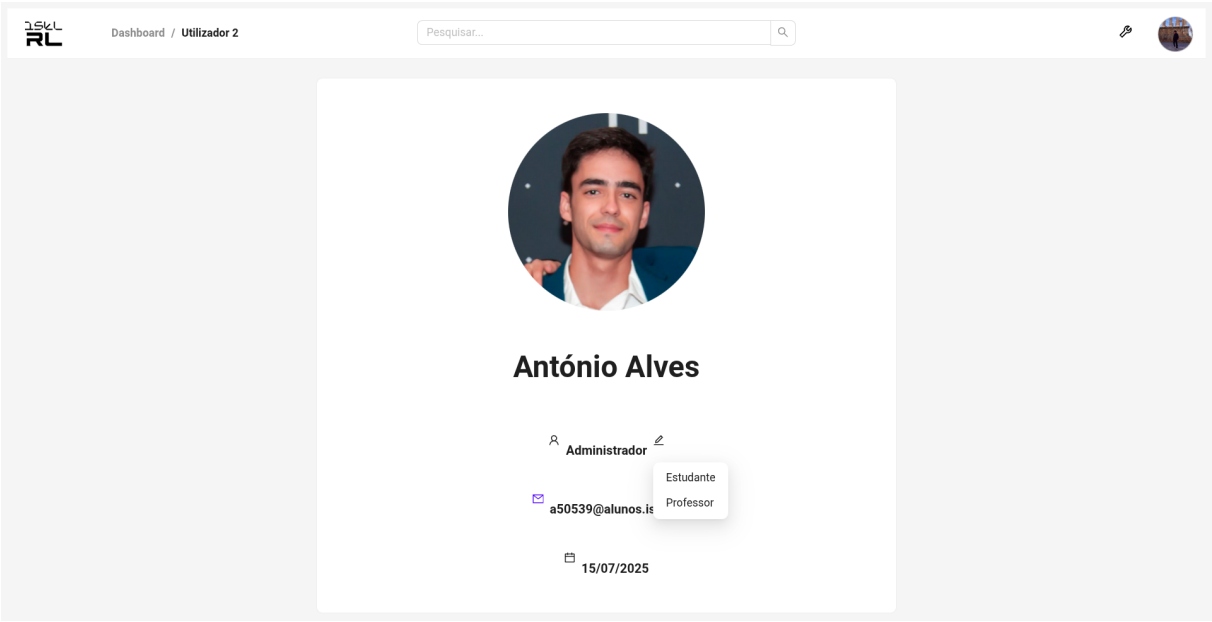


Figure 5.20: Role change dropdown (admin)

5.6 Summary

The web application leverages the power and flexibility of Next.js to deliver a secure, scalable, and user-centric platform for remote laboratory access. Its architecture ensures clear separation of concerns, robust authentication, and a responsive user experience, while its integration with modern deployment practices guarantees maintainability and future growth. For further details on the features and implementation of the web application, this chapter should be used as the main reference, in conjunction with the official Next.js documentation [15].

Chapter 6

Project Organization and Deployment

The Remote Lab project is publicly available at
`https://github.com/isel-remote-lab/remote-lab`.
Detailed deployment instructions and further documentation can also be found in the project's GitHub Wiki [51]. To run the application with the full configuration, access to the `private/` submodule is required, as it contains essential API keys and secret files. This submodule is private for security reasons; therefore, contributors must request access to the "Remote Lab" GitHub organization or directly to the private repository to obtain the necessary credentials and configuration files. Both the backend (`api/`) and frontend (`website/`) repositories maintain separate `main` and `develop` branches to support stable releases and ongoing development, respectively. This branching strategy facilitates collaborative development and continuous integration while ensuring production stability.

6.1 Project Structure

The Remote Lab project is organized into several main directories, most of which are managed as GitHub submodules. This modular approach enables independent development, versioning, and access control for each core component, supporting both scalability and security. Submodules also facilitate collaboration among different teams and ensure that sensitive information is handled appropriately.

The main submodules and directories are:

- **api/** – Contains the backend source code (Kotlin, Spring Boot). Responsible for business logic, user management, and laboratory session control.

- **db/** – Includes database scripts and documentation, supporting the system's persistence layer.

- **website/** – Holds the frontend web application (Next.js, React). Provides the user interface for laboratory access, scheduling, and management.
- **docker/** – Contains Docker-related scripts and files for deployment and automation.
- **nginx/** – Provides NGINX configuration files for reverse proxying, with separate configs for development and production (`default.dev.conf`, `default.prod.conf`).
- **docs/** – Stores project documentation, including technical reports, user guides, and architectural diagrams.
- **img/** – Contains project images, such as diagrams, screenshots, and other visual assets.
- **private/** – Dedicated to sensitive files and configurations, such as environment variables and secrets. This submodule is not included directly in the main repository, ensuring that only authorized members have access to confidential information like API keys and external service credentials. Shared secrets are located in `private/shared/secrets/.env`.
- **wiki/** – Stores the GitHub Wiki pages, including project documentation, deployment instructions, and other relevant information.
- **hardware/** – Contains code and configuration for integrating and simulating hardware devices, enabling interaction with real or mock laboratory equipment.

This structure allows for independent development, testing, and deployment of each component. The addition of the `docker/` directory and the clear separation of NGINX configs reflect the evolution towards more robust DevOps and automation practices.

6.1.1 Hardware Integration and Simulation

The **hardware/** directory contains code and configuration for simulating or integrating real hardware devices with the Remote Lab platform. This component enables the platform to interact with physical laboratory equipment or to provide a mock hardware service for development and demonstration purposes.

A dedicated Dockerfile is provided in `hardware/` to containerize the hardware service. The `docker-compose.yml` file includes a `hardware-example` service, which can be started as part of a demonstration or integration scenario. This service communicates with the backend API using secure API keys and environment variables defined in the `private/` submodule.

To run the hardware service as part of the platform, ensure the following steps:

1. Ensure the `private/` submodule is initialized and contains the required API keys and secrets.
2. Use Docker Compose with the appropriate profile (e.g., `demo`) to start the hardware service alongside the other components:

```
docker compose --profile demo up --build
```

3. Alternatively, use the `start.sh` script with the correct flags to include the hardware service in the deployment.

4. The hardware service will be accessible and integrated with the backend, allowing for remote interaction, testing, or demonstration of hardware-related features.

This modular approach allows the platform to be extended with real or simulated hardware, supporting both educational and research scenarios.

At this stage, WebSocket connections are used in the website (frontend) to communicate with the API, enabling real-time, bidirectional features such as live updates and interactive laboratory sessions. The hardware service can also use WebSocket to interact with the backend for device integration and simulation.

It is also possible to run the hardware service outside of Docker by executing the JavaScript code directly on the intended hardware device. This approach provides flexibility for real-world deployments, allowing the integration of physical laboratory equipment without requiring containerization, and enabling the service to run natively on devices such as single-board computers or embedded systems.

6.2 Deployment

The deployment process for the Remote Lab platform is designed to be straightforward, secure, and reproducible, leveraging modern Development and Operations (DevOps) practices and containerization technologies [52, 53].

6.2.1 Containerization and Orchestration

All major components—including the backend (API), frontend (website), and database—are containerized using Docker [52]. Docker Compose orchestrates multi-container deployments, manages networking between services, and handles environment-specific configurations. Recent updates introduced profiles in `docker-compose.yml` (`api`, `full`, `prod`, `dev`, `cloudflare`, `demo`), allowing flexible deployments adapted to different scenarios. The `hardware-example` service was added to simulate real hardware in demonstration environments.

The frontend now provides separate Dockerfiles for development and production (`Dockerfile.dev`, `Dockerfile.prod`), optimizing builds and facilitating debugging.

6.2.2 Environment Configuration and Secrets

Sensitive configuration files and environment variables are managed in the **private/** submodule, specifically in `private/shared/secrets/.env`. This file is automatically loaded by the startup

script, ensuring that the required credentials and secrets are available to all services.

6.2.3 Automation with start.sh

To streamline deployment, the platform provides a `start.sh` script at the repository root. This script offers a flexible and automated way to initialize and manage the entire environment, supporting a variety of deployment scenarios through command-line flags. Its main features include:

- Selection of environment: development (`-dev` or `-d`) or production (`-prod` or `-p`).
- API-only mode (`-api` or `-a`) to start only the backend service for development or testing.
- Demo mode (`-demo` or `-dm`) to enable demonstration features, including the hardware simulation service.
- Cloudflare tunnel integration (`-cloudflare` or `-c`) for secure remote access.
- Branch switching (`-switch` or `-s`) to automatically check out the appropriate branch (e.g., `main`, `develop`, or `demo`) in submodules.
- Log viewing (`-logs` or `-l`) to display container logs without restarting services.

The script automatically loads environment variables and secrets from `private/shared/secrets/.env`, and, in production, from `private/frontend/.env`. It dynamically selects Docker Compose profiles based on the chosen flags, enabling or disabling services such as the frontend, hardware simulation, and Cloudflare tunnel as needed. The script also supports combinations of flags for highly customizable deployments, such as running the full stack in demo mode with remote access, or starting only the API for local development.

This automation ensures that all required services, secrets, and configurations are correctly initialized, significantly reducing manual setup and the risk of misconfiguration.

6.2.4 Cloudflare Tunneling

Cloudflare Tunnel is integrated into the automation workflow via `start.sh` and Docker Compose. The `cloudflared` service can be activated via profile, making secure remote access simpler and more automated. This allows development or demonstration environments to be securely exposed to the internet without the need to open firewall ports.

6.2.5 Nginx as Reverse Proxy

NGINX acts as a reverse proxy to route HTTP requests to both the frontend (website) and backend (API) services [54]. The configuration files now distinguish between development and production, allowing security and performance to be optimized according to the context. NGINX is automatically included in the Docker Compose setup.

6.2.6 Deployment Steps

1. **Clone the Repository and Submodules:** Clone the main repository and initialize all submodules, including `private/`, to ensure all components and configurations are available.

2. **Configure Environment Variables:** Ensure that all required environment variables and secret files are present in the appropriate locations, as provided by the `private/` submodule.

3. **Build and Start Services:** Use the provided `docker-compose.yml` file to build and start all services with a single command (e.g., `docker compose up --build`), or simply run the `start.sh` script at the project root, which automates the entire process including building images, starting containers, and loading environment variables and secrets.

4. **Access the Platform:** Once all containers are running, the platform can be accessed via the configured web address. NGINX is used as a reverse proxy to route traffic securely to the appropriate services [54].

6.3 Build and Continuous Integration and Continuous Deployment (CI/CD)

- **Gradle:** Used for building and managing backend dependencies.
- **pnpm:** Used for frontend dependency management and builds, replacing npm for greater speed and reliability.
- **Dockerfiles:** Multi-stage builds are used for both backend and frontend to optimize image size and security with Docker. The frontend provides separate Dockerfiles for dev/prod.
- **GitHub Actions:** The frontend provides CI/CD pipelines for the `main` and `develop` branches, performing lint, format, check, build, and automatically committing lint/format fixes. The backend is built via Gradle and Docker and can be easily integrated into CI/CD pipelines.

6.4 Summary

The implemented infrastructure leverages modern web technologies, containerization, and modular design to provide a robust, scalable, and maintainable platform for remote laboratory access [15, 52]. The deployment process is automated, secure, and ready for future growth, ensuring that the platform can evolve to meet new requirements and increased demand. Recent improvements in automation, environment management, and CI/CD pipelines further strengthen the project's readiness for collaborative development and production deployment.

Chapter 7

Experimental Results

To validate the proper behavior of the system, comprehensive unit tests were implemented for the web API. Each module undergoes thorough testing, covering both success and error states to ensure robustness and reliability.

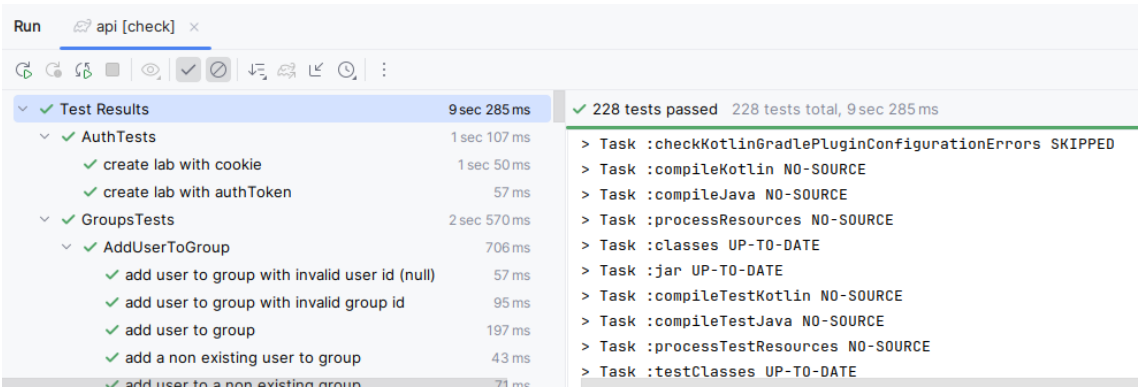


Figure 7.1: Unit Tests Results

In future work, we aim to expand the test coverage significantly and implement load testing to evaluate system performance under various conditions. These load tests will provide valuable insights into response times during peak usage periods, enabling us to identify bottlenecks and optimize system performance accordingly. The results from these comprehensive tests will guide further improvements to enhance the overall system efficiency and user experience.

Chapter 8

Conclusions

8.1 Conclusions

This report has presented the design, implementation, and validation of a comprehensive platform for remote laboratory access, addressing the growing need for flexible and accessible laboratory resources in educational and research environments. The solution successfully bridges the gap between traditional hands-on experimentation and the requirements of modern remote learning and collaboration.

The developed platform successfully fulfills all primary objectives established at the project's inception. The implementation of a robust web API provides comprehensive management capabilities for users, laboratories, and hardware resources. The intuitive web application interface ensures accessibility for users with varying technical backgrounds, while the secure authentication and authorization mechanisms, coupled with role-based access control, guarantee system integrity and appropriate resource allocation.

The well-designed database system ensures reliable data persistence, supporting the platform's scalability requirements. Furthermore, the implementation of remote manipulation capabilities of laboratory devices enables real-time interaction with experimental equipment, maintaining the essential hands-on experience that characterizes quality laboratory work.

8.2 Future Work

While the current implementation successfully addresses the primary objectives, several areas present opportunities for future enhancement. The expansion of the testing framework to include comprehensive load testing will provide valuable insights into system performance under high-usage conditions. This analysis will inform optimization strategies and guide infrastructure scaling decisions.

Future development efforts should focus on expanding hardware compatibility to support a broader range of laboratory equipment types. Additionally, the implementation of advanced monitoring and analytics capabilities could provide valuable usage insights and facilitate resource optimization. The integration of collaborative features, such as shared experimentation sessions

and real-time collaboration tools, would further enhance the platform's educational value.

Furthermore, the development of mobile applications and the enhancement of accessibility features would broaden the platform's reach and usability across diverse user populations.

References

[1] Baeldung. Handlerinterceptors vs. filters in spring mvc. https://www.baeldung.com/spring-mvc-handlerinterceptor-vs-filter. Accessed: 2025-07-15.

[2] MIT iLab Project. ilab shared architecture (isa). https://icampus.mit.edu/projects/ilabs/index.html, 2025. Accessed: 2025-05-29.

[3] LabShare Project. Labshare: Collaborative remote laboratories. https://dbpedia.org/page/Labshare, 2025. Accessed: 2025-05-29.

[4] WebLab-Deusto. Weblab-deusto: Remote laboratory for electronics. https://www.weblab.deusto.es/, 2025. Accessed: 2025-05-29.

[5] L. F. Zapata Rivera and Larrondo Petrie. The remote laboratory management system (rlms) pattern. https://hillside.net/sugarloafplop/2018/program/papers/GroupA/12.3.pdf, 2018. Accessed: 2025-05-30.

[6] D. Lowe, T. Machet, et al. Sahara labs: Remote laboratory framework. https://github.com/sahara-labs, 2013. University of Technology Sydney. Accessed: 2025-05-30.

[7] Wikipedia. Role-based access control. https://en.wikipedia.org/wiki/Role-based_access_control. Accessed: 2025-07-14.

[8] Wikipedia. Rest. https://en.wikipedia.org/wiki/REST. Accessed: 2025-07-14.

[9] Wikipedia. Uniform resource identifier. https://en.wikipedia.org/wiki/Uniform_Resource_Identifier. Accessed: 2025-07-14.

[10] Wikipedia. Http (hypertext transfer protocol). https://en.wikipedia.org/wiki/HTTP. Accessed: 2025-07-14.

[11] Spring. Spring framework. https://spring.io/projects/spring-framework. Accessed: 2025-06-01.

[12] Jetbrains. Kotlin programming language. https://kotlinlang.org/. Accessed: 2025-06-01.

[13] Spring. Spring. https://spring.io/. Accessed: 2025-06-01.

[14] The PostgreSQL Global Development Group. Postgresql. https://www.postgresql.org/. Accessed: 2025-06-01.

[15] Vercel. Next.js documentation. https://nextjs.org/docs, 2025. Accessed: 2025-05-30.

[16] Meta Open Source. React. https://react.dev/reference/react. Accessed: 2025-06-01.

[17] Vercel. Authentication. https://nextjs.org/docs/app/building-your-application/authentication, 2025. Accessed: 2025-05-30.

[18] Microsoft. Microsoft oauth. https://learn.microsoft.com/en-us/entra/architecture/auth-oauth2. Accessed: 2025-06-01.

[19] Wikipedia. Create, read, update and delete. https://en.wikipedia.org/wiki/Create,_read,_update_and_delete. Accessed: 2025-07-15.

[20] Spring. The ioc container. https://docs.spring.io/spring-framework/docs/3.2.x/spring-framework-reference/html/beans.html. Accessed: 2025-06-01.

[21] Wikipedia. Strategy pattern. https://en.wikipedia.org/wiki/Strategy_pattern. Accessed: 2025-06-01.

[22] Spring. Spring depedency injection. https://docs.spring.io/spring-framework/reference/core/beans/dependencies/factory-collaborators.html. Accessed: 2025-06-01.

[23] Oracle. Java programming language. https://www.java.com/en/. Accessed: 2025-06-01.

[24] Spring. Interface beanfactory. https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/beans/factory/BeanFactory.html. Accessed: 2025-06-01.

[25] Spring. Annotated controllers. https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-controller.html. Accessed: 2025-06-01.

[26] Spring. Filters. https://docs.spring.io/spring-framework/reference/web/webmvc/filters.html. Accessed: 2025-07-11.

[27] Spring. Interface handlerinterceptor. https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/web/servlet/HandlerInterceptor.html. Accessed: 2025-06-01.

[28] Spring. Dispatcherservlet. https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-servlet.html. Accessed: 2025-07-11.

[29] Json. Introducing json. https://www.json.org/json-en.html. Accessed: 2025-07-15.

[30] Jetbrains. Kotlin serialization. https://kotlinlang.org/docs/serialization.html. Accessed: 2025-06-01.

[31] Jdbi. Jdbi 3 developer guide. https://jdbi.org/. Accessed: 2025-06-01.

[32] Oracle. Java jdbc api. https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/. Accessed: 2025-06-01.

[33] Rossen Stoyanchev. Sseemitter. https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/web/servlet/mvc/method/annotation/SseEmitter.html. Accessed: 2025-07-11.

[34] HTML Living Standard. Server-sent events. https://html.spec.whatwg.org/multipage/server-sent-events.html. Accessed: 2025-07-11.

[35] Jetbrains. Kotlin coroutines. https://kotlinlang.org/docs/coroutines-overview.html. Accessed: 2025-07-11.

[36] Redis. Redis. https://redis.io/. Accessed: 2025-07-11.

[37] Spring. Spring data redis. https://spring.io/projects/spring-data-redis. Accessed: 2025-07-15.

[38] Postman. Postman. https://www.postman.com/. Accessed: 2025-06-01.

[39] RL Dev Team. Remote lab public api. https://documenter.getpostman.com/view/31383926/2sB2cVgNBA. Accessed: 2025-06-01.

[40] RL Dev Team. Remote lab private api. https://documenter.getpostman.com/view/31383926/2sB2qUm4cA. Accessed: 2025-06-01.

[41] Ant Design Team. Ant design - a ui design language and react ui library. https://ant.design/, 2025. Accessed: 2025-06-01.

[42] Vercel. App router: Routing, layouts, and pages. https://nextjs.org/docs/app/building-your-application/routing, 2025. Accessed: 2025-05-30.

[43] Vercel. Server and client components. https://nextjs.org/docs/app/building-your-application/rendering/server-and-client-components, 2025. Accessed: 2025-05-30.

[44] Vercel. Data fetching. https://nextjs.org/docs/app/building-your-application/data-fetching, 2025. Accessed: 2025-05-30.

[45] Vercel. Api routes. https://nextjs.org/docs/app/building-your-application/routing/api-routes, 2025. Accessed: 2025-05-30.

[46] Juan Cruz Martinez. Using next.js server actions to call external apis. https://auth0.com/blog/using-nextjs-server-actions-to-call-external-apis/, 2023. Accessed: 2025-05-29.

[47] Vercel. Loading ui and streaming. https://nextjs.org/docs/app/building-your-application/routing/loading-ui-and-streaming, 2025. Accessed: 2025-05-30.

[48] xterm.js Contributors. xterm.js - terminal for the web. https://xtermjs.org/, 2025. Accessed: 2025-06-01.

[49] IETF. The websocket protocol. RFC 6455, 2011. Accessed: 2025-06-01.

[50] NextAuth.js Team. Nextauth.js - authentication for next.js. https://next-auth.js.org/, 2025. Accessed: 2025-06-01.

[51] RL Dev Team. Remote lab project wiki. https://github.com/isel-remote-lab/remote-lab/wiki, 2025. Accessed: 2025-06-01.

[52] Docker Inc. Docker documentation. https://docs.docker.com/, 2025. Accessed: 2025-05-30.

[53] Docker Inc. Docker compose documentation. https://docs.docker.com/compose/, 2025. Accessed: 2025-05-30.

[54] Inc. NGINX. Nginx documentation. https://docs.nginx.com/, 2025. Accessed: 2025-05-30.

Appendix A

Database Documentation

A.1 Introduction

This document provides a comprehensive overview of the database structure, including its entities, attributes, and relationships. It also details key implementation decisions.

A.2 Database Overview

The database has been designed using an Entity-Relationship (ER) approach. This methodology enables a clear understanding of how different entities interact within the system. The following section presents the ER model diagram.

The database is implemented using PostgreSQL and has been tested with sample data in a Docker container.

The next section presents the ER model in detail.

A.3 Entity-Relationship Model

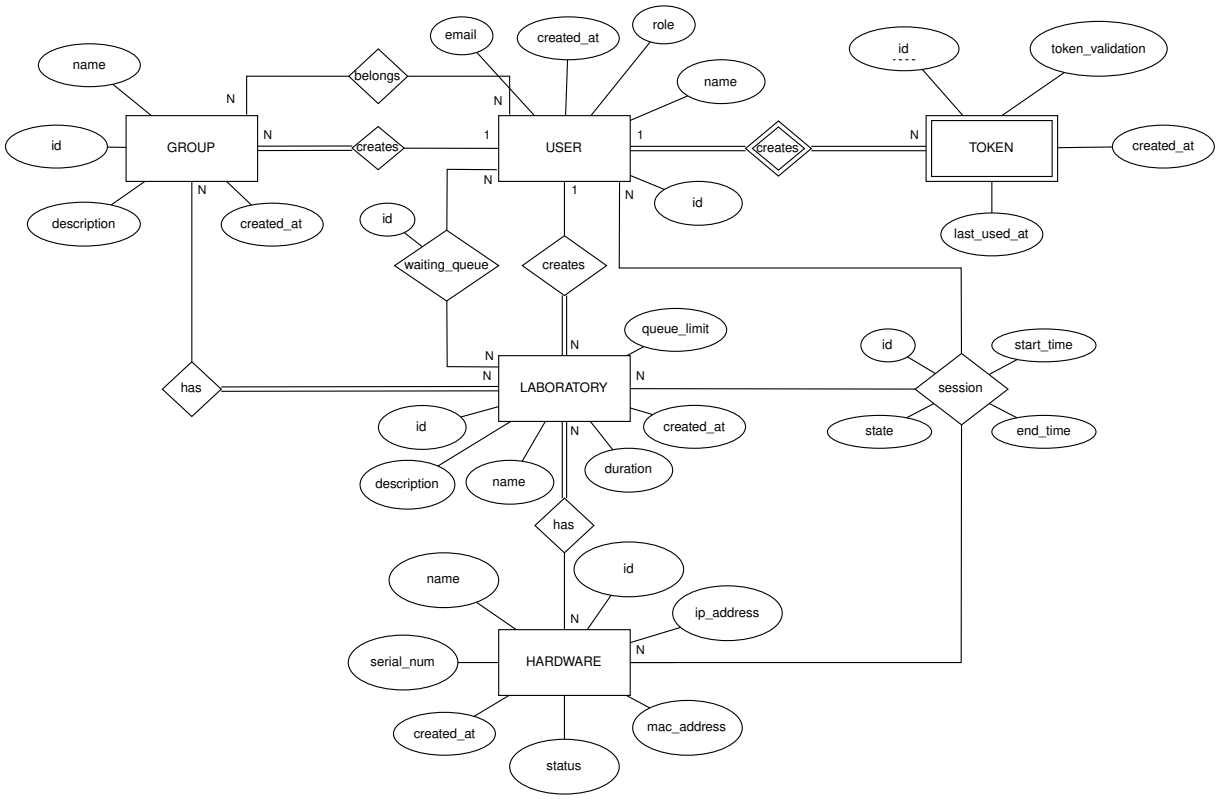


Figure A.1: ER Model

A.4 Entities and Attributes

This section provides a comprehensive description of each entity and its attributes.

A.4.1 User

The **User** entity is fundamental to the database and represents system users.

It has an **id** as its primary key, implemented as an identity column. An identity column automatically generates values from an implicit sequence. Therefore, whenever a new user is created, a unique id is automatically generated. The id is an integer data type, chosen to simplify database queries by using a simple numeric identifier instead of email or username.

The entity includes character sequence attributes: **name** and **email**. All these fields are required (not null), and the email must be unique. Initially, the name is automatically set to match the user's name returned by OAuth login. While names are used for display purposes and may not be unique, emails serve as unique identifiers for specific tasks since they are unique. The specific constraints for these attributes are defined in the application domain.

As a discriminator attribute, this entity contains a **role**. This role determines the user's role within the system. This attribute is a char, and the possible options are S (Student), P

(Professor), *A* (Administrator).

Additionally, it includes a **created_at** field as a required timestamp.

A.4.2 Token

Token is implemented as a weak entity since it cannot exist independently and requires a user_id for identification. Each token is associated with a user and is essential for authentication purposes. Users must have a valid token when interacting with the system.

As a weak entity, it requires a partial key, **id**, generated automatically. It also contains a **token_validation** field. This attribute stores a randomly generated hash created by the system and is used to verify token validity. For instance, when a user's token is included in an authorization header, the system compares it with this stored value during each interaction. It is implemented as a required character sequence.

The entity also includes **created_at** and **last_used_at** timestamps, both required fields. These attributes enable token validity checking based on timing rules defined by the application domain.

A.4.3 Group

The **Group** entity represents various types of user groupings, such as student classes, work groups, or professor groups.

Each group has an **id** as its primary key, implemented as an auto-generated integer identity column. This unique identifier facilitates efficient querying and group identification.

The **name** is a required character sequence that stores the user-defined name for the group. Group names can be modified after creation, with specific length and format restrictions defined by the application domain.

Groups can include an optional **description** stored as a text type. This field allows users to provide detailed information about the group's purpose or characteristics. While technically unlimited in length, practical limitations may be imposed by the application domain.

The entity also includes a **created_at** timestamp, which can be used for analytical and administrative purposes.

A.4.4 Laboratory

The **Laboratory** entity represents physical or virtual laboratory spaces within the system.

Each laboratory has an **id** as its primary key, implemented as an auto-generated integer identity column. This unique identifier simplifies laboratory identification and database queries.

The **name** attribute is a required character sequence that stores the laboratory's designation. Specific naming conventions and restrictions are defined by the application domain.

The entity includes a **duration** attribute, which specifies the standard duration of laboratory sessions in minutes. This integer field is required and helps manage session scheduling.

Additionally, it includes a **queue_limit** to control the number of users in the waiting queue and a **created_at** timestamp for tracking laboratory creation and administrative purposes.

A.4.5 Hardware

The **Hardware** entity represents physical equipment within the laboratory system, such as computers or FPGAs.

Each piece of hardware is identified by an **id** primary key, implemented as an auto-generated integer identity column to facilitate equipment tracking and queries.

The entity includes a required **name** attribute as a character sequence for equipment identification.

A required **serial_num** character sequence stores the hardware's serial number for inventory management.

The **ip_address** and **mac_address** are character sequence fields that may be null depending on the hardware type, as not all equipment requires network connectivity.

A required **status** character sequence tracks the current state of the hardware (e.g., available, occupied).

Like other entities, it includes a **created_at** timestamp for inventory tracking and administrative purposes.

A.5 Associations

This section details the relationships between entities, explaining how they interact within the system and what restrictions they have.

User associations include:

- **Token** - A weak association where each user may have multiple tokens (N:1). Tokens are automatically generated by the system as needed, with the user's primary key serving as part of the token's composite key. It is worth noting that an Administrator can also hold the position of Professor due to hierarchical considerations.
- **Group** - Users have two distinct relationships with groups:
  - **creates** - Users can create multiple groups, but each group has exactly one creator (N:1). Only Professors can create groups.
  - **belongs** - Users can belong to multiple groups, and groups can have multiple members (N:N)
- **Laboratory** - Users have three types of laboratory associations:
  - **creates** - Users can create multiple laboratories, but each laboratory has one creator (N:1). Only Professors can create laboratories.

- **joins** - Users can join multiple laboratories over time, and laboratories can be joined by multiple users (N:N). While the application domain may impose restrictions (e.g., limiting concurrent laboratory access), the database structure maintains this flexibility to support historical tracking.
- **session** - Users can have multiple sessions. The exact number of concurrent sessions is defined in the application domain. This association also includes the hardware entity, since they are related to each other. This association contains three important attributes: **start_time** and **end_time**, both as timestamps for session control and statistics, and a **state** attribute. The state attribute is a char type and can be P (In Progress), C (Completed), or S (Scheduled).

Group has an additional association with Laboratory, controlling laboratory access permissions. Groups can be associated with multiple laboratories, and laboratories can be accessible to multiple groups (N:N).

Laboratory maintains these additional associations:

- **Lab Session** - Laboratories can have multiple sessions, but each session is associated with exactly one laboratory (N:1)
- **Hardware** - Laboratories can be assigned multiple pieces of hardware, and hardware can be assigned to multiple laboratories over time (N:N)

Note that additional constraints and business rules are implemented at the web API level. Please refer to the web API documentation for detailed information about these restrictions.

Appendix B

Domain Configurations

```
{
  "user": {
    "tokenSizeInBytes": 16,
    "tokenTtl": 24,
    "tokenRollingTtl": 1,
    "tokenTtlDurationUnit": "HOURS",
    "maxTokensPerUser": 3
  },
  "laboratory": {
    "name": {
      "min": 3,
      "max": 100,
      "optional": false
    },
    "description": {
      "min": 10,
      "max": 1000,
      "optional": true
    },
    "duration": {
      "min": 1,
      "max": 480,
      "optional": false,
      "unit": "MINUTES"
    },
    "queueLimit": {
      "min": 1,
      "max": 100,
      "optional": false
    }
  },
  "group": {
    "name": {
      "min": 3,
      "max": 100,
      "optional": false
    },
    "description": {
      "min": 10,
      "max": 1000,
      "optional": true
    }
  },
  "hardware": {
    "name": {
```

```
      "min": 3,
      "max": 100,
      "optional": false
    },
    "serialNumber": {
      "min": 1,
      "max": 100,
      "optional": false
    },
    "status": {
      "min": 1,
      "max": 1,
      "availableOptions": ["A", "O", "M"],
      "optional": false
    },
    "macAddress": {
      "min": 12,
      "max": 12,
      "optional": true
    },
    "ipAddress": {
      "min": 1,
      "max": 30,
      "optional": true
    }
  }
}
```

Listing B.1: Domain Configurations